

TTK4900 – Engineering Cybernetics, Master's Thesis

Biologically plausible alternatives to backpropagation for training neural networks

Mats Wiig Martinussen

*Supervisor:
Bjarne André Grimstad
Trondheim, May 2026*



Department of Engineering Cybernetics, NTNU 2026

Table of Contents

List of Figures	iii
List of Tables	iii
1 Introduction	1
1.1 Background and Motivation	1
1.1.1 Recent Progress in Artificial Intelligence	1
1.1.2 Open Challenges in Current AI Systems	1
1.1.3 Two Perspectives on the Path Toward General Intelligence	2
1.1.4 Biological Systems as a Reference for General Intelligence	2
1.1.5 Biological Inspiration: Architecture and Learning	3
1.1.6 Motivation for Alternative Learning Rules	3
1.1.7 Knowledge Gaps	3
1.2 Contribution, Research Objectives and Research Questions	4
1.2.1 Contribution	4
1.2.2 Objectives	4
1.2.3 Research Questions	4
1.3 Structure of the Thesis	4
2 Theory	4
2.1 How the brain learns	5
2.1.1 Hebbian learning	5
2.1.2 Neuromodulators and three-factor learning	6
2.1.3 Stochasticity in biological learning	6
2.2 Learning in artificial neural networks	7
2.2.1 Feedforward neural networks and supervised learning	7
2.2.2 Backpropagation	7
2.2.3 Why backpropagation is biologically implausible	9
2.3 Biologically plausible alternatives to backpropagation	9
2.4 Weight perturbation	11
2.5 Node perturbation	13
2.6 Existing comparisons of weight and node perturbation	17
2.7 Literature gap	18
3 Original Theoretical Contribution	18

3.1	Overview of the contribution	18
3.2	Pre-activation noise induced by WP	19
3.3	Input-scaled node perturbation	20
3.4	Expected equivalence between WP and IS-NP	21
3.5	Variance reduction theorem	23
3.6	Why IS-NP may improve over existing NP variants	25
3.7	Summary of theoretical contribution	26
4	Method	27
4.1	Empirical evaluation strategy	27
4.2	Learning problems and datasets	28
4.2.1	Synthetic sinus regression	28
4.2.2	California housing regression	28
4.2.3	MNIST classification	28
4.2.4	CIFAR-10 classification	29
4.2.5	Scaled-input diagnostics	29
4.3	Model architecture and initialization	31
4.4	Algorithms	33
4.5	Evaluation metrics	36
4.5.1	Training and test performance	36
4.5.2	Directional alignment	37
4.5.3	Estimator variance	38
4.5.4	Aggregate Ranking of Perturbation Methods	39
4.6	Hyperparameter selection and experimental protocol	40
5	Results	43
5.1	Sinus Regression	43
5.2	California Housing Regression	46
5.3	MNIST Classification	49
5.4	CIFAR-10 Classification	51
5.5	Aggregate Ranking of Perturbation Methods	53
5.6	Scaled Input Sensitivity	53
6	Discussion	54
6.1	Overview of the main findings	54
6.2	Practical capability and limits of perturbation-based learning	54

6.3	Empirical support for IS-NP	55
6.4	Why input scaling matters	56
6.5	Limitations	57
6.6	Future work	57
7	Conclusion	58
	Appendix	59
A	Checkpoint-Specific Estimator Diagnostics	59
A.1	Sinus Regression	59
A.2	California Housing Regression	61
A.3	MNIST Classification	63
A.4	CIFAR-10 Classification	65

List of Figures

1	Results for the sinus regression task.	44
2	Results for the California housing regression task.	47
3	Results for the MNIST classification task.	49
4	Results for the CIFAR-10 classification task.	51
5	Effect of scaling the sinus input range from $[-1, 1]$ to $[-5, 5]$	53
6	Checkpoint-specific cosine similarity for the sinus regression task.	60
7	Checkpoint-specific estimator variance for the sinus regression task.	60
8	Checkpoint-specific cosine similarity for the California housing regression task. . .	62
9	Checkpoint-specific estimator variance for the California housing regression task. .	62
10	Checkpoint-specific cosine similarity for the MNIST classification task.	64
11	Checkpoint-specific estimator variance for the MNIST classification task.	64
12	Checkpoint-specific cosine similarity for the CIFAR-10 classification task.	66
13	Checkpoint-specific estimator variance for the CIFAR-10 classification task. . . .	66

List of Tables

1	Comparison of perturbation-based update rules in the literature.	16
2	Learning problems used in the empirical evaluation.	30
3	Model architectures used in the empirical evaluation.	32
4	Training and estimator-diagnostic settings for each learning problem.	36

5	Local hyperparameter grids used during tuning. For perturbation methods, the grid is the Cartesian product of the listed learning rates and perturbation scales. The selected setting is the final configuration used for the thesis results and is shown as the central value of the displayed local grid.	42
6	Summary metrics for the sinus regression task. Values are mean $\pm$ standard deviation across seeds.	45
7	Metric-wise ranking of perturbation methods for the sinus regression task. Lower ranks are better.	46
8	Summary metrics for the California housing regression task. Values are mean $\pm$ standard deviation across seeds.	48
9	Metric-wise ranking of perturbation methods for the California housing regression task. Lower ranks are better.	49
10	Summary metrics for the MNIST classification task. Values are mean $\pm$ standard deviation across seeds.	50
11	Metric-wise ranking of perturbation methods for the MNIST classification task. Lower ranks are better.	51
12	Summary metrics for the CIFAR-10 classification task. Values are mean $\pm$ standard deviation across seeds.	52
13	Metric-wise ranking of perturbation methods for the CIFAR-10 classification task. Lower ranks are better.	53
14	Aggregate metric-wise ranking of perturbation methods across the four learning problems. Values are average ranks across tasks, and lower values are better. . . .	53

1 Introduction

1.1 Background and Motivation

1.1.1 Recent Progress in Artificial Intelligence

Artificial intelligence has undergone a dramatic transformation over the past decade, emerging as one of the most powerful technological forces of our time. Recent advances in deep learning and large-scale optimization have produced systems capable of performing tasks once considered exclusive to human intelligence. Large language models can generate coherent text, write code, and assist in complex reasoning tasks. Reinforcement learning systems have achieved superhuman performance in games such as chess and Go, while breakthroughs in scientific machine learning have enabled progress on long-standing challenges such as protein structure prediction. At the same time, autonomous systems are transitioning from research prototypes to real-world deployment, with self-driving vehicles now operating in urban environments. Taken together, these developments suggest that artificial intelligence is not only progressing rapidly, but may be approaching a level of capability that raises fundamental questions about the feasibility of artificial general intelligence.

1.1.2 Open Challenges in Current AI Systems

Despite this rapid progress, there is increasing debate within the research community regarding whether current approaches are sufficient to achieve artificial general intelligence. Yann LeCun, often referred to as one of the “godfathers of AI,” has been particularly vocal in his criticism of current paradigms, recently co-founding a company backed by over \$1 billion to pursue fundamentally different approaches to intelligence. These developments suggest that, despite impressive empirical success, fundamental challenges may still remain. Among others, these include:

- **Energy intensity.** The 10^{11} neurons and 10^{15} synapses contained in the typical human brain expend approximately 20 W of power, whereas a digital simulation of an artificial neural network of approximately the same size consumes 7.9MW. Source: Wong, T. M. et al. 1014. Report no. RJ10502 (ALM1211-004) (IBM, 2012). The power consumption of this simulation of the brain puts that of conventional digital systems into context. You can also include a graph that show the energy consumption per model since 1985. Exponential graph. Cant keep growing like this forever. So maybe right now, the real bottleneck to intelligence is energy. So we need more energy efficient architectures to achieve AGI.
- **Lack of continual learning.** Most modern AI systems lack the ability to learn continuously from experience. During training, models are optimized on a fixed dataset, after which their parameters remain largely static during deployment, preventing them from naturally adapting to new information. While updates through fine-tuning are possible, these often suffer from *catastrophic forgetting*, where newly acquired knowledge interferes with previously learned information. This limitation is problematic because a generally intelligent system should be able to incorporate new information as it becomes available—for example, adapting to a specific user or immediately integrating major real-world events—without degrading prior knowledge. Current approaches such as in-context learning partially address this by conditioning on recent inputs, but this remains a temporary and indirect workaround rather than true persistent learning. As such, the lack of robust continual learning mechanisms represents a key limitation of current AI systems.
- **Limited out-of-distribution generalization.** Current AI systems often perform well on tasks that resemble their training data, but their performance can degrade significantly when faced with novel situations. This is because most models rely on the assumption that training and test data follow similar distributions, an assumption that rarely holds in real-world settings. As a result, models tend to exploit statistical regularities or spurious correlations present in the training data, rather than learning underlying structure that transfers to new

environments. Recent work has shown that even when models appear robust on standard benchmarks, significant failure modes can emerge under distribution shifts, indicating that current evaluation methods may overestimate true generalization performance. Since general intelligence fundamentally requires the ability to apply knowledge to new and unseen situations, limited out-of-distribution generalization represents a central challenge for current AI systems.

- **Limited abstraction.** Current AI systems often struggle to learn high-level, abstract concepts that generalize across different contexts. Instead, they tend to rely on statistical regularities and surface-level features present in the training data, rather than capturing the underlying meaning of a concept. For example, a model trained to recognize chairs may rely on features such as the presence of four legs, and fail to identify unconventional designs that do not match these patterns, despite clearly serving the same purpose. In contrast, the abstract concept of a chair is simply something to sit on, independent of its specific appearance. As a result, models may perform well on familiar inputs but fail to generalize when superficial characteristics change.
- **Lack of grounding in the physical world.** Most modern AI systems learn from static datasets rather than through interaction with the environment, limiting their ability to connect representations to real-world experience. As a result, their knowledge is largely derived from correlations in data rather than direct interaction with the underlying phenomena. For example, a language model may learn about cooking by reading large numbers of recipes, but it has no direct experience of taste, texture, or smell, and therefore lacks a grounded understanding of what the concepts it describes actually correspond to in the real world. This makes their behavior highly dependent on the data they were trained on and limits their ability to reason about concepts that are inherently tied to physical interaction.

1.1.3 Two Perspectives on the Path Toward General Intelligence

There are broadly two perspectives on the trajectory of current AI systems. On the one hand, some researchers argue that existing approaches may be fundamentally limited, lacking key ingredients required for general intelligence. On the other hand, there is a belief that continued scaling of data, model size, and computational resources will lead to increasingly general capabilities, with no clear evidence of saturation so far.

Regardless of which perspective proves correct, systems that achieve general intelligence must exhibit the capabilities outlined above, such as continual learning, strong generalization, and the ability to form abstract and grounded representations. Even if these properties ultimately emerge from scaling alone, it remains unclear whether current approaches represent the most efficient or principled way of achieving them. The natural thing to do is, therefore, to look at systems that already exhibit these capabilities.

1.1.4 Biological Systems as a Reference for General Intelligence

Biological systems provide a proof of existence that the capabilities outlined above can be achieved in practice. Humans and other animals exhibit continual learning, adapting to new experiences without requiring a separate training phase. They demonstrate strong generalization, applying knowledge to novel situations—for example, recognizing objects under new conditions or solving problems they have not explicitly encountered before.

In addition, biological learning is highly sample-efficient. A child can learn the meaning of a new word, such as “butterfly,” from only a few exposures by leveraging contextual cues, such as where a speaker is looking or pointing, rather than requiring repeated observations across large datasets. Biological systems also form abstract representations, allowing them to recognize that an object is a chair regardless of its specific appearance, as long as it serves the function of something to sit on.

Finally, biological intelligence is grounded in interaction with the physical world. Concepts such as “heavy” or “taste” are not learned through descriptions alone, but through direct sensory experience and interaction. Together, these properties highlight that the challenges identified in current AI systems are routinely addressed in biological systems, suggesting that they may provide a valuable source of inspiration for developing more general and adaptive learning mechanisms.

1.1.5 Biological Inspiration: Architecture and Learning

Research inspired by biological systems broadly follows two directions. One approach focuses on developing architectures that more closely resemble biological neural systems. This includes neuromorphic hardware **I could show an example or two here, and generally spend some more time on neuromorphic architectures**, but also more radical approaches where biological neurons themselves are used as the computational substrate. A particularly striking example is the CL1 system developed by Cortical Labs, where networks of living neurons are interfaced with a digital environment and trained to perform tasks through real-time interaction. In such systems, neural activity both drives and responds to external inputs, forming a closed feedback loop that allows the system to adapt its behavior over time. Despite consisting of relatively simple neural cultures, these systems have demonstrated the ability to learn goal-directed behavior, highlighting the potential of biological substrates for information processing and learning. While still in an early stage, this line of work represents a fundamentally different paradigm from conventional AI, where computation is no longer confined to silicon, but emerges from the dynamics of living neural systems.

An alternative approach is to focus not on replicating the full biological architecture, but on understanding and emulating the mechanisms by which such systems learn. Rather than building biological systems directly, this direction seeks to identify learning rules that capture key properties of biological learning. In this thesis, we focus on this second direction, and in particular on biologically plausible learning algorithms.

1.1.6 Motivation for Alternative Learning Rules

Backpropagation is not biologically plausible. It relies on mechanisms such as exact gradient computation, symmetric weight transport, and the propagation of global error signals through the network—none of which are known to be implementable in biological neural systems. If architectures that more closely resemble biological systems, such as CL1, turn out to be a viable path toward more general intelligence, then backpropagation is no longer applicable as a learning mechanism. In such settings, alternative learning rules are not just interesting, but necessary.

At the same time, even within current machine learning systems, backpropagation is not without limitations. It is computationally and memory intensive, and its effectiveness relies on highly structured training procedures and large amounts of data. This raises the question of whether alternative learning mechanisms, potentially inspired by biological systems, can offer different trade-offs or point toward new directions for improving existing approaches.

1.1.7 Knowledge Gaps

Although perturbation-based learning methods are attractive from a biological perspective, several questions remain open. Existing work has shown that weight perturbation and node perturbation can provide useful learning signals, but the relationship between these methods is not fully understood in nonlinear multilayer networks. In particular, it is unclear whether the advantage of node perturbation comes only from perturbing a lower-dimensional space, or whether there are deeper estimator-level differences between weight-level and activity-level perturbations.

There is also limited empirical evidence comparing perturbation-based learning rules across supervised tasks of increasing complexity using the same network class and evaluation procedure. As a result, it remains unclear where these methods work well, where they begin to fail, and whether

theoretically motivated variants can improve their practical performance. These gaps motivate the focus of this thesis: to review the current perturbation-learning literature, develop a more direct theoretical comparison between relevant methods, and evaluate the resulting learning rules empirically.

1.2 Contribution, Research Objectives and Research Questions

1.2.1 Contribution

1.2.2 Objectives

The objective of this thesis is to investigate perturbation-based learning methods as biologically plausible alternatives to backpropagation. The thesis combines literature analysis, theoretical development, and empirical evaluation.

- **Establish the current state of perturbation-based learning.** Review and compare existing perturbation-based learning methods, with emphasis on how different approaches inject noise, form learning signals, and update weights. This provides the foundation for identifying what is already understood, which existing variants are most relevant, and where the literature remains incomplete.
- **Further develop the theory of perturbation-based learning.** Build on the existing literature to clarify the relationship between common perturbation algorithms, and use this understanding to develop a theoretically motivated variant of node perturbation. The aim is to strengthen the theoretical basis of perturbation learning and identify a modification that may improve estimator quality and learning performance.
- **Evaluate perturbation-based methods empirically.** Assess how existing perturbation-based learning methods and the proposed method perform across supervised learning tasks of increasing complexity. The goal is to understand both when these methods work well and when they begin to break down, using backpropagation as the standard reference method.

1.2.3 Research Questions

1.3 Structure of the Thesis

2 Theory

Chapter 1 motivated why biologically plausible learning algorithms are interesting. To understand why perturbation methods are relevant in this context, we need to establish two things: first, that the current dominant training algorithm, backpropagation, is difficult to reconcile with biological constraints; and second, that perturbation-based methods satisfy these constraints more naturally.

This chapter therefore begins by introducing the biological learning principles relevant for the thesis, drawing on work on synaptic plasticity, neuromodulation, and three-factor learning rules (Hebb, 1949; Schultz et al., 1997; Frémaux and Gerstner, 2016; Gerstner et al., 2018). It then reviews learning in artificial neural networks, derives the backpropagation update (Rumelhart et al., 1986), and explains why backpropagation is considered biologically implausible (Crick, 1989; Lillicrap et al., 2020). The chapter then turns to perturbation-based learning and addresses the first research objective: to establish the current state of perturbation-based learning. This includes deriving weight perturbation and node perturbation, reviewing important variants of these methods (Williams, 1992; Werfel et al., 2005; Hiratani et al., 2022; Züge et al., 2023; Dalm et al., 2024), comparing how the literature relates WP and NP, and identifying the remaining gaps that motivate the theoretical contribution in Chapter 3.

2.1 How the brain learns

Learning can be understood as the process by which the brain builds and updates internal models of the world. Dehaene (2021) describes learning as constructing such models from experience. These models may represent, for example, the layout of a neighbourhood, the way our limbs respond when we move, or the rules of a language we use in daily life. They are not fixed at birth, but gradually formed through interaction with the environment. In this sense, learning means changing the brain’s internal representations so that they better reflect the structure of the world.

These internal representations are implemented in networks of neurons connected by synapses. Neurons communicate through brief electrical spikes, and the probability that a neuron spikes depends on the input it receives from other neurons (Kandel et al., 2021). Synapses determine how strongly one neuron influences another. A strong synapse makes activity in the presynaptic neuron more likely to affect the postsynaptic neuron, while a weak synapse has little influence. When large numbers of neurons interact, their joint activity can encode structured information, such as the sound of a familiar word or the spatial structure of a known street. These learned representations are therefore not stored as explicit symbolic rules, but as distributed patterns of neural activity shaped by synaptic connections.

The question is then how these internal models change. Dehaene also describes learning as “adjusting the parameters of a mental model” (Dehaene, 2021). In the brain, the natural analogue of such parameters is the strength of synaptic connections. Synapses are plastic: their strengths can increase or decrease depending on experience (Kandel et al., 2021). When synaptic strengths change, the influence neurons have on each other also changes, and this modifies the activity patterns the network can produce. Learning in the brain can therefore be viewed as the adjustment of synaptic connections in a large neural network, so that the representations encoded by the network become better adapted to the environment.

There are many biological mechanisms that can modify synaptic strengths, and neuroscience contains several theories of how such mechanisms give rise to learning. The purpose of this section is not to give a complete account of biological plasticity, but to introduce the principles that are most relevant for the learning algorithms studied in this thesis. No single theory fully explains how the brain learns, but the ideas discussed below represent influential mechanisms that have motivated biologically grounded learning algorithms, including perturbation-based learning.

2.1.1 Hebbian learning

Hebbian learning is a central idea in theories of synaptic plasticity. The basic principle, introduced by Hebb (1949), is that a synapse should be strengthened when activity in the presynaptic neuron consistently helps activate the postsynaptic neuron. Abstractly, this can be written as

$$\Delta w_{ij} = \eta f(x_i) f(x_j),$$

where w_{ij} denotes the connection strength from neuron i to neuron j , x_i and x_j denote their activities, $f(\cdot)$ allows the update to depend nonlinearly on neural activity, and $\eta > 0$ sets the learning rate. The important point is that this is a local rule: the synapse does not need access to global information about the task or the network, but only to signals associated with the two neurons it connects.

This type of local learning can still produce meaningful structure. Since Hebbian updates strengthen connections between neurons that are active together, they reinforce statistical regularities in the input. Oja’s rule provides a classic example of this idea in a stabilized form: by adding a normalization-like correction to the basic Hebbian mechanism, it allows a single neuron to learn the first principal component of the input distribution (Oja, 1982). Hebbian learning therefore shows that purely local, unsupervised plasticity can support nontrivial forms of representation learning.

However, the limitation of the learning rule is that this rule contains no feedback about the outcome of the network state. The synapse changes when pre- and postsynaptic activity are correlated, but

the update does not depend on whether the resulting action, prediction, or behaviour was good or bad. It therefore cannot by itself explain how the brain learns from reward, error, success, or failure. For this reason, Hebbian plasticity is an important foundation for synaptic learning, but not a complete account of learning in the brain.

2.1.2 Neuromodulators and three-factor learning

To learn from success and failure, the brain needs a feedback signal in addition to local synaptic activity. One important class of such signals is provided by neuromodulators. Unlike ordinary point-to-point synaptic communication, neuromodulatory signals can be broadcast to larger brain regions, allowing many neurons and synapses to receive the same information. Dopamine is a central example. Activity in midbrain dopamine neurons has been associated with reward-prediction errors: it tends to increase when outcomes are better than expected and decrease when outcomes are worse than expected (Schultz et al., 1997). Dopamine can therefore act as a global scalar feedback signal, carrying information about whether recent neural activity or behaviour led to a better or worse outcome than expected.

This leads to three-factor learning rules, where synaptic change depends on a local presynaptic term, a local postsynaptic term, and a global modulatory signal:

$$\Delta w_{ij} = \eta f_i(x_i) f_j(x_j) M. \quad (1)$$

Here, w_{ij} is the synaptic weight from neuron i to neuron j , $f_i(x_i)$ and $f_j(x_j)$ are potentially nonlinear functions of the pre- and postsynaptic states, M is a global modulatory signal, and η is a learning-rate parameter. The local term $f_i(x_i) f_j(x_j)$ can be interpreted as an eligibility trace: a synapse-specific signal determined by recent pre- and postsynaptic activity. Intuitively, the global signal M carries information about whether the outcome was good or bad, while the local eligibility trace indicates how relevant that synapse was for producing the activity that led to the outcome.

Three-factor learning rules overcome the core limitation of pure Hebbian plasticity: they allow learning to depend not only on local neural activity, but also on whether the resulting behaviour or prediction was successful. This makes three-factor rules a biologically grounded form of supervised or reinforcement learning, and they are widely considered a plausible mechanism for outcome-driven synaptic change in the brain (Frémaux and Gerstner, 2016; Gerstner et al., 2018).

2.1.3 Stochasticity in biological learning

Biological learning also takes place in a noisy environment. Neural firing is variable, synaptic transmission is stochastic, and the same stimulus does not always produce exactly the same neural response. Rather than being only a nuisance, this stochasticity can provide a source of exploration. If random fluctuations in neural activity or synaptic efficacy are correlated with later reward or error signals, then the nervous system can reinforce fluctuations that improve performance and suppress those that do not (Seung, 2003). This idea directly motivates perturbation-based learning algorithms, where random perturbations are introduced into weights or neural activities and then correlated with a global scalar learning signal.

The biological principles relevant for this thesis can therefore be summarized as follows. The learning rules considered here use information from the presynaptic neuron, the postsynaptic neuron, and a global scalar feedback signal, such as a reward, error, or neuromodulatory signal. In addition, stochastic fluctuations in weights or neural activity can provide the exploration needed for learning. Together, pre- and postsynaptic information, global feedback, and stochastic perturbations form the biological motivation for the perturbation-based learning rules studied in the remainder of this chapter.

2.2 Learning in artificial neural networks

2.2.1 Feedforward neural networks and supervised learning

An artificial neural network consists of layers of artificial neurons connected by weights. In a feedforward network, information moves in one direction: from the input layer, through one or more hidden layers, and finally to the output layer. Each neuron in a layer receives signals from neurons in the previous layer, and the weights determine how strongly each previous neuron influences it. In this sense, weights play the same abstract role as synapses in biological neural networks. For each neuron, the incoming activations are combined using these weights and a bias term, before the result is passed through a nonlinear activation function. Following the standard notation for feedforward networks (Goodfellow et al., 2016), this can be written as

$$z_j^\ell = \sum_i W_{ij}^\ell x_i^{\ell-1} + b_j^\ell, \quad x_j^\ell = \phi(z_j^\ell). \quad (2)$$

Here, z_j^ℓ is the pre-activation of neuron j in layer ℓ , $x_i^{\ell-1}$ is the activity of neuron i in the previous layer, W_{ij}^ℓ is the corresponding weight, b_j^ℓ is a bias, and $\phi(\cdot)$ is the activation function. Repeating this operation over many layers gives a flexible parameterized function from inputs to outputs. Despite the simplicity of this construction, such networks are highly expressive. The universal approximation theorem shows that feedforward networks can approximate any continuous function on a compact domain, provided that the network is sufficiently large (Hornik, 1991). The setting considered here is supervised learning. We are given examples $(\mathbf{x}, \mathbf{y})$, where $\mathbf{x}$ is an input and $\mathbf{y}$ is the desired output. A network with parameters $\boldsymbol{\theta}$, collecting all weights and biases, produces a prediction $\hat{\mathbf{y}}_{\boldsymbol{\theta}}(\mathbf{x})$. The quality of this prediction is measured by a loss function $L(\mathbf{y}, \hat{\mathbf{y}}_{\boldsymbol{\theta}}(\mathbf{x}))$. The aim is not simply to reduce the loss on the observed training examples, but to learn parameters that also perform well on unseen examples drawn from the same underlying distribution. This generalization objective can be written as

$$\boldsymbol{\theta}^* = \arg \min_{\boldsymbol{\theta}} \mathbb{E}_{(\mathbf{x}, \mathbf{y})} [L(\mathbf{y}, \hat{\mathbf{y}}_{\boldsymbol{\theta}}(\mathbf{x}))]. \quad (3)$$

In principle, learning would amount to solving this optimization problem and directly finding $\boldsymbol{\theta}^*$. For modern neural networks, however, this is not tractable. The parameter space is large, the loss landscape is generally non-convex, and the full data-generating distribution is unknown. Instead, learning is performed through iterative updates. Starting from an initial parameter vector, a learning algorithm repeatedly chooses an update direction and takes a small step:

$$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \Delta \boldsymbol{\theta}.$$

The practical problem is therefore to determine how the update $\Delta \boldsymbol{\theta}$ should be computed. This gives a direct connection to Dehaene’s description of learning as ‘adjusting the parameters of a mental model’ (Dehaene, 2021). In the brain, the relevant parameters are synaptic strengths. In artificial neural networks, they are the weights and biases in $\boldsymbol{\theta}$. Both cases can therefore be viewed as learning by changing the parameters of a network so that its behaviour better matches the structure of the task or environment.

At this level, biological and artificial learning look similar: both involve modifying connection strengths in a network. The difference appears when we look at how these modifications are computed. In modern artificial neural networks, this is usually done using backpropagation (Rumelhart et al., 1986). We therefore next derive the backpropagation update and show why it breaks with the biological learning principles introduced above.

2.2.2 Backpropagation

Backpropagation (Rumelhart et al., 1986) is the standard algorithm for training neural networks. Its purpose is to minimize the objective in Eq. (3) by updating the parameters in the direction of decreasing loss. For a single input–target pair, this requires the gradient

$$\nabla_{\boldsymbol{\theta}} L(\mathbf{y}, \hat{\mathbf{y}}_{\boldsymbol{\theta}}(\mathbf{x})), \quad (4)$$

which tells us how each parameter should be changed in order to reduce the loss. Backpropagation computes this gradient exactly by applying the chain rule backward through the network. Starting from the output layer, error information is propagated layer by layer toward the input, so that each weight receives a gradient describing how it contributed to the final loss.

The derivation begins at the final layer because the loss depends directly on the network output. Any effect that earlier layers have on the loss must pass through the later layers. Therefore, once we know how the loss changes with respect to the output activations, we can obtain the gradients for earlier layers by repeatedly applying the chain rule backward through the network.

The first quantity to compute is the derivative of the loss with respect to each output activation:

$$\frac{\partial L}{\partial x_j^\ell}.$$

Using the activation relation from Eq. (2), this can be converted into a derivative with respect to the corresponding pre-activation. We define this quantity as the error term

$$\delta_j^\ell := \frac{\partial L}{\partial z_j^\ell} = \frac{\partial L}{\partial x_j^\ell} \phi'(z_j^\ell). \quad (5)$$

The term δ_j^ℓ measures how sensitive the loss is to a small change in the pre-activation z_j^ℓ . This is the starting point for the backward propagation of error signals.

From the feedforward definition,

$$z_j^\ell = \sum_i W_{ij}^\ell x_i^{\ell-1} + b_j^\ell,$$

the derivative of z_j^ℓ with respect to an incoming weight is

$$\frac{\partial z_j^\ell}{\partial W_{ij}^\ell} = x_i^{\ell-1}.$$

Applying the chain rule then gives the gradient of the loss with respect to this weight:

$$\begin{aligned} \frac{\partial L}{\partial W_{ij}^\ell} &= \frac{\partial L}{\partial z_j^\ell} \frac{\partial z_j^\ell}{\partial W_{ij}^\ell} \\ &= \delta_j^\ell x_i^{\ell-1}. \end{aligned} \quad (6) \quad (7)$$

Similarly, since the pre-activation depends linearly on the bias,

$$\frac{\partial z_j^\ell}{\partial b_j^\ell} = 1, \quad \frac{\partial L}{\partial b_j^\ell} = \delta_j^\ell.$$

These expressions are not special to the output layer. For any layer ℓ , once the error term δ_j^ℓ is known, the gradients of the incoming weight and bias are given by

$$\frac{\partial L}{\partial W_{ij}^\ell} = x_i^{\ell-1} \delta_j^\ell, \quad \frac{\partial L}{\partial b_j^\ell} = \delta_j^\ell.$$

Thus, the forward activations $x_i^{\ell-1}$ and the backward error terms δ_j^ℓ are sufficient to compute the parameter gradients in that layer.

The remaining question is how to compute δ_j^ℓ for hidden layers. In the output layer, the loss depends directly on the activations, so the error term can be computed from the loss. In a hidden layer, however, the loss does not depend directly on x_j^ℓ . A hidden activation affects the loss only through its influence on the next layer. Applying the chain rule gives

$$\frac{\partial L}{\partial x_j^\ell} = \sum_k \frac{\partial L}{\partial z_k^{\ell+1}} \frac{\partial z_k^{\ell+1}}{\partial x_j^\ell} = \sum_k W_{jk}^{\ell+1} \delta_k^{\ell+1},$$

where the sum runs over all neurons k in the next layer. Multiplying by the derivative of the activation function then gives the recursive backpropagation equation

$$\delta_j^\ell = \frac{\partial L}{\partial z_j^\ell} = \left(\sum_k W_{jk}^{\ell+1} \delta_k^{\ell+1} \right) \phi'(z_j^\ell).$$

This recursion explains why the algorithm proceeds backward. To compute the error term in layer ℓ , the error terms in layer $\ell + 1$ must already be known. Backpropagation therefore starts at the output layer and recursively propagates error terms backward through the network. Once this backward pass has been completed, the exact gradient $\partial L / \partial \theta$ is available for all parameters.

Substituting the recursive expression for δ_j^ℓ into the weight-gradient formula makes the full structure of a hidden-layer update explicit:

$$\frac{\partial L}{\partial W_{ij}^\ell} = x_i^{\ell-1} \phi'(z_j^\ell) \sum_k W_{jk}^{\ell+1} \delta_k^{\ell+1}. \quad (8)$$

This expression shows that the gradient for a hidden-layer weight depends on three factors: the presynaptic activation $x_i^{\ell-1}$, the local activation derivative $\phi'(z_j^\ell)$, and a weighted sum of error terms from the next layer.

Backpropagation is powerful because it computes the exact gradient of the loss with respect to every parameter, using one forward pass and one backward pass. This makes it the natural reference method for the empirical evaluation in this thesis. In a sense, the gradient that perturbation-based methods try to estimate from random perturbations and scalar feedback is exactly the quantity that backpropagation computes directly. However, as suggested by the biological learning principles discussed above, the mechanism used by backpropagation differs substantially from how learning is believed to occur in the brain. This is the issue we turn to next.

2.2.3 Why backpropagation is biologically implausible

The biological difficulty becomes clear from the hidden-layer gradient derived above:

$$\frac{\partial L}{\partial W_{ij}^\ell} = x_i^{\ell-1} \phi'(z_j^\ell) \sum_k W_{jk}^{\ell+1} \delta_k^{\ell+1}.$$

The first factor, $x_i^{\ell-1}$, is presynaptic activity, and the activation derivative $\phi'(z_j^\ell)$ is locally available at the postsynaptic neuron. These terms are therefore at least compatible with local synaptic learning. The final factor, however, is different. It requires a precise, signed, neuron-specific error signal from the next layer, weighted by the downstream connections $W_{jk}^{\ell+1}$. This is not information available at the synapse, nor is it a global scalar feedback signal of the kind discussed in three-factor learning. A literal implementation of backpropagation would therefore require biological circuits to transmit exact error derivatives backward through the network, with well-known additional difficulties such as the weight transport problem, where the backward pathway must use the exact transpose of the forward weights (Crick, 1989; Lillicrap et al., 2020). These requirements make standard backpropagation highly difficult to reconcile with known biological learning mechanisms.

This does not reduce the importance of backpropagation as an artificial learning algorithm. It remains powerful precisely because it computes the gradient that perturbation-based methods can only estimate. However, its biological implausibility motivates the search for alternative learning rules that use more biologically realistic ingredients. This is the motivation for the biologically plausible alternatives considered next.

2.3 Biologically plausible alternatives to backpropagation

The biological implausibility of strict backpropagation has motivated a broad range of alternative learning rules. These methods differ in how closely they try to approximate backpropagation and

in which biological constraints they prioritize. Some aim to preserve much of the efficiency of backpropagation while relaxing specific assumptions, such as exact symmetric feedback weights. Others avoid explicit backward error propagation altogether, instead relying on local objectives, recurrent dynamics, or stochastic exploration. The purpose of this section is not to review all such methods in detail, but to briefly place perturbation-based learning within the broader landscape of biologically plausible alternatives.

Feedback alignment changes how error information is sent to hidden layers. In standard backpropagation, the error signal is propagated backward using the exact transpose of the forward weight matrices. Feedback alignment replaces these exact backward weights with fixed random feedback weights. Surprisingly, the network can still learn, because the forward weights adapt so that the random feedback signals become useful for reducing the loss (Lillicrap et al., 2016). Direct feedback alignment takes this idea further by sending the output error directly to each hidden layer through fixed random feedback connections, rather than propagating the error recursively from layer to layer (Nøkland, 2016). These methods are biologically relevant because they relax the weight transport problem: the backward pathway no longer needs to contain an exact copy of the forward weights. However, they still rely on relatively detailed, vector-valued error signals being delivered to hidden layers, and therefore remain closer to backpropagation than to scalar-feedback learning rules.

Predictive coding takes a different approach. Instead of explicitly sending gradients backward through the network, each layer tries to predict the activity of another layer, and learning is driven by the mismatch between predictions and actual activity. In the original predictive coding framework, feedback connections carry predictions from higher to lower cortical areas, while feedforward connections carry residual prediction errors (Rao and Ballard, 1999). Later work showed that, under certain assumptions, these local prediction errors can approximate the error signals used by backpropagation, allowing synaptic updates to be written in a more local Hebbian-like form (Whittington and Bogacz, 2017). Predictive coding is therefore attractive from a biological perspective because it uses local prediction errors and recurrent interactions, which are more natural in cortical circuits than a separate backward gradient pass. At the same time, it typically requires structured recurrent dynamics and an inference process before learning, making it more complex than simple scalar-feedback rules.

The forward-forward algorithm removes the backward pass entirely. Instead of one forward pass followed by one backward pass, it uses two forward passes: one with positive data and one with negative data (Hinton, 2022). Each layer has its own local objective, usually based on a measure of “goodness” such as the squared activity in that layer. The weights are updated so that positive examples produce high goodness and negative examples produce low goodness. This makes the method more local than backpropagation, since layers can be trained using their own objectives without propagating derivatives backward through the network. However, the method also changes the learning problem: it depends on how positive and negative examples are constructed, and it does not directly estimate the gradient of a single global supervised loss.

Perturbation methods take a different and conceptually simpler approach. Instead of computing an error derivative for each neuron or synapse, they inject random perturbations into either the weights or the neural activities and observe how these perturbations affect performance. If a perturbation improves performance, the corresponding change is reinforced; if it worsens performance, the change is reversed. This gives learning rules that combine local information about the perturbation and neural activity with a global scalar feedback signal, making them closely related to three-factor learning principles. Weight perturbation applies this idea at the level of synaptic weights, while node perturbation applies it at the level of neural activity (Williams, 1992; Werfel et al., 2005; Hiratani et al., 2022). Perturbation methods are generally less efficient than backpropagation, but they are attractive because they require only forward evaluations, stochastic fluctuations, and scalar feedback.

This thesis focuses on perturbation methods for three reasons. First, they align naturally with the biological principles introduced earlier: local synaptic information, stochastic neural activity or synaptic fluctuations, and global modulatory feedback. Second, they are simple enough to implement and analyze in standard neural network models, while still being plausible candidates

for more biologically motivated architectures. Third, the literature contains fewer general theoretical comparisons between different perturbation methods than between backpropagation-like alternatives. Rather than surveying many biologically plausible algorithms superficially, this thesis therefore focuses on one family of methods in depth. The remainder of this chapter reviews weight perturbation, node perturbation, and existing comparisons between them, before identifying the literature gap that motivates the original theoretical contribution in Chapter 3.

Having established why perturbation-based learning is a relevant family of biologically motivated alternatives to backpropagation, we now turn to the research objectives of the thesis. The following sections address Objective 1 by reviewing the main perturbation methods in the literature and identifying what remains unresolved. This establishes the current state of perturbation-based learning and leads naturally to Objective 2, which is addressed in Chapter 3 by building on the literature review to motivate and develop the node perturbation variant proposed in this thesis.

2.4 Weight perturbation

Weight perturbation applies perturbation-based learning directly in parameter space by adding random noise to the network weights and biases and observing how the resulting perturbed network affects the loss. For example, if a weight has value 1.00, a sampled perturbation of 0.07 gives the perturbed weight 1.07. If the resulting forward pass gives a lower loss than some baseline, the update should move the weight in the same direction, for example from 1.00 to 1.03. If the perturbed network performs worse, the update should instead move in the opposite direction. In this way, random parameter fluctuations are correlated with a scalar learning signal to produce a gradient estimate. This is the basic idea behind weight perturbation, but there are several ways to formalize it. Below, we derive the score-function update rule used in the rest of this thesis. At the end of the subsection, we briefly discuss another common formulation, the finite-difference perturbation rule.

In this section, we formalize weight perturbation using the score-function estimator. This estimator is also known as the likelihood-ratio gradient estimator in the Monte Carlo simulation literature (Glynn, 1990). In neural networks and reinforcement learning, the same log-derivative principle is closely associated with the REINFORCE algorithm of Williams (1992), which derives unbiased gradient-following rules from stochastic perturbations and scalar feedback. The resulting multiply-by-perturbation update rule is also used in much of the recent perturbation-learning literature considered here, including Werfel et al. (2005), Hiratani et al. (2022), Züge et al. (2023), and Dalm et al. (2024). These works motivate and analyze the rule in different ways, for example through small-perturbation limits, expected gradient alignment, learning-curve analysis, or directional-derivative arguments. Here, we derive the update rule explicitly from the score-function identity. This provides a clean way to view weight perturbation as an unbiased Monte Carlo estimator of the gradient of an expected loss under parameter perturbations.

Let $\boldsymbol{\mu}$ denote the deterministic parameter vector that we seek to optimize, consisting of all weights and biases in the network. Instead of minimizing the deterministic loss $L(\boldsymbol{\mu})$ directly, we introduce a distribution over perturbed parameter vectors,

$$\boldsymbol{\theta} \sim p_{\boldsymbol{\mu}}(\boldsymbol{\theta}) = \mathcal{N}(\boldsymbol{\mu}, \sigma^2 I),$$

where $\sigma > 0$ is a fixed perturbation scale. A single draw from this distribution perturbs the full parameter vector at once, meaning that all weights and biases are perturbed simultaneously rather than one parameter at a time. Perturbing one parameter at a time would give a more isolated estimate of each parameter’s effect, but would require one or more additional forward evaluations per parameter and is therefore impractical for large networks. The parallel approach introduces more randomness into each update, but greatly reduces the number of required forward passes. This trade-off is standard in simultaneous perturbation methods, most notably simultaneous perturbation stochastic approximation (Spall, 1992), and is also the form used in the recent WP-NP literature considered here. The objective is then the expected loss under this distribution,

$$F(\boldsymbol{\mu}) := \mathbb{E}_{\boldsymbol{\theta} \sim p_{\boldsymbol{\mu}}} [L(\boldsymbol{\theta})].$$

Minimizing this expectation means choosing $\boldsymbol{\mu}$ so that the distribution $p_{\boldsymbol{\mu}}$ places high probability on parameter values with low loss and low probability on parameter values with high loss. Conceptually, this is similar to ordinary loss minimization, but it is expressed in a stochastic setting where the loss is averaged over random perturbations. In the limit $\sigma \rightarrow 0$, the distribution collapses to a point mass at $\boldsymbol{\mu}$, and the expected loss reduces to the deterministic loss $L(\boldsymbol{\mu})$. We now seek to construct an estimator of $\nabla_{\boldsymbol{\mu}} F(\boldsymbol{\mu})$ using only forward evaluations of perturbed networks. To compute $\nabla_{\boldsymbol{\mu}} F(\boldsymbol{\mu})$, we write the expectation explicitly as

$$F(\boldsymbol{\mu}) = \int L(\boldsymbol{\theta}) p_{\boldsymbol{\mu}}(\boldsymbol{\theta}) d\boldsymbol{\theta},$$

and differentiate under the integral sign:

$$\nabla_{\boldsymbol{\mu}} F(\boldsymbol{\mu}) = \int L(\boldsymbol{\theta}) \nabla_{\boldsymbol{\mu}} p_{\boldsymbol{\mu}}(\boldsymbol{\theta}) d\boldsymbol{\theta}.$$

This step is valid under standard regularity conditions, and allows us to make the dependence on $\boldsymbol{\mu}$ explicit inside the integral. Following the score-function estimator (Williams, 1992), we use the identity

$$\nabla_{\boldsymbol{\mu}} p_{\boldsymbol{\mu}}(\boldsymbol{\theta}) = p_{\boldsymbol{\mu}}(\boldsymbol{\theta}) \nabla_{\boldsymbol{\mu}} \log p_{\boldsymbol{\mu}}(\boldsymbol{\theta}),$$

which yields

$$\nabla_{\boldsymbol{\mu}} F(\boldsymbol{\mu}) = \mathbb{E}_{\boldsymbol{\theta} \sim p_{\boldsymbol{\mu}}} [L(\boldsymbol{\theta}) \nabla_{\boldsymbol{\mu}} \log p_{\boldsymbol{\mu}}(\boldsymbol{\theta})]. \quad (9)$$

This expression rewrites the gradient of an expectation as an expectation involving the score function $\nabla_{\boldsymbol{\mu}} \log p_{\boldsymbol{\mu}}(\boldsymbol{\theta})$, enabling gradient estimation using Monte Carlo sampling.

For the Gaussian distribution $p_{\boldsymbol{\mu}}(\boldsymbol{\theta}) = \mathcal{N}(\boldsymbol{\mu}, \sigma^2 \mathbf{I})$, the log-density is given by

$$\log p_{\boldsymbol{\mu}}(\boldsymbol{\theta}) = -\frac{1}{2\sigma^2} \|\boldsymbol{\theta} - \boldsymbol{\mu}\|^2 + C,$$

and its gradient is

$$\nabla_{\boldsymbol{\mu}} \log p_{\boldsymbol{\mu}}(\boldsymbol{\theta}) = \frac{\boldsymbol{\theta} - \boldsymbol{\mu}}{\sigma^2}.$$

Substituting into (9) yields

$$\nabla_{\boldsymbol{\mu}} F(\boldsymbol{\mu}) = \mathbb{E} \left[L(\boldsymbol{\theta}) \frac{\boldsymbol{\theta} - \boldsymbol{\mu}}{\sigma^2} \right].$$

Using the reparameterization $\boldsymbol{\theta} = \boldsymbol{\mu} + \sigma \boldsymbol{\xi}$, we obtain

$$\nabla_{\boldsymbol{\mu}} F(\boldsymbol{\mu}) = \mathbb{E} \left[L(\boldsymbol{\mu} + \sigma \boldsymbol{\xi}) \frac{\boldsymbol{\xi}}{\sigma} \right]. \quad (10)$$

In practice, the expectation in (10) cannot be computed exactly, as it requires integrating over all possible perturbations. Instead, we approximate it using Monte Carlo sampling, which allows us to estimate the gradient using only forward evaluations of the network. For a single sample $\boldsymbol{\xi}$, this yields the estimator

$$\hat{g}_i = L(\boldsymbol{\mu} + \sigma \boldsymbol{\xi}) \frac{\xi_i}{\sigma}.$$

To reduce the variance of this estimator, one can subtract a baseline b that does not depend on the individual perturbation ξ_i , following the standard baseline argument used in REINFORCE (Williams, 1992). The estimator remains unbiased, since

$$\mathbb{E} \left[b \frac{\xi_i}{\sigma} \right] = 0,$$

as ξ_i has zero mean and b is independent of ξ_i . One example is a leave-one-out batch-mean baseline: if several perturbations are sampled in the same batch, the baseline for one perturbation can be formed from the average loss of the other perturbations. Intuitively, such a baseline centers the learning signal: perturbations that lead to lower-than-average loss reinforce their contribution,

while those that perform worse than average are reversed. This reduces variance in the gradient estimate without affecting its expectation. The resulting estimator becomes

$$\hat{g}_i = (L(\boldsymbol{\mu} + \sigma \boldsymbol{\xi}) - b) \frac{\xi_i}{\sigma}. \quad (11)$$

Applying gradient descent, we update the parameters in the negative gradient direction:

$$\Delta \mu_i = -\eta (L(\boldsymbol{\mu} + \sigma \boldsymbol{\xi}) - b) \frac{\xi_i}{\sigma}, \quad (12)$$

where η is the learning rate. This corresponds to the weight perturbation update rule. This update rule constitutes an unbiased Monte Carlo estimator of the gradient of the expected loss, implying that its expected update direction coincides with the true backpropagation gradient, although potentially with high variance.

This makes WP biologically plausible in the sense discussed above. The update does not require a neuron-specific error derivative, but only a stochastic perturbation of the parameter and a scalar feedback signal measuring whether the perturbed network performed better or worse than the baseline. The term $L(\boldsymbol{\mu} + \sigma \boldsymbol{\xi}) - b$ therefore plays a role similar to the global modulatory signal M in Eq. (1), while ξ_i represents the local stochastic fluctuation being reinforced or suppressed. Thus, WP fits the idea that random fluctuations in synaptic efficacy can be correlated with later reward or error signals to drive learning.

Other forms of weight perturbation have also been proposed in the literature. Earlier perturbation rules were often formulated as finite-difference methods, where the change in error caused by a small perturbation is used to approximate a derivative. For example, in the weight perturbation method of Flower and Jabri (1992), a weight perturbation ϵ_{ij} is applied and the resulting change in error is used to form an estimate proportional to $\Delta \ell / \epsilon_{ij}$. This differs from the score-function-style estimator derived above, where the scalar learning signal is multiplied by the sampled perturbation rather than divided by it. Finite-difference perturbation rules are therefore closely related to weight perturbation, but belong to a different estimator family. Since the recent WP-NP literature most relevant to this thesis, including the works discussed above, primarily uses the multiply-by-perturbation form, and since the theoretical analysis in Chapter 3 builds on this estimator structure, the remainder of the thesis focuses on the score-function-style estimator derived above.

2.5 Node perturbation

Node perturbation provides an alternative to weight perturbation by applying noise at the level of neural activity rather than directly to each individual parameter. Recall that the pre-activation of neuron j in layer ℓ is

$$z_j^\ell = \sum_i W_{ij}^\ell x_i^{\ell-1} + b_j^\ell.$$

Instead of perturbing each incoming weight W_{ij}^ℓ and bias b_j^ℓ separately, node perturbation perturbs z_j^ℓ directly. The resulting scalar learning signal is then assigned back to the incoming weights using local presynaptic activity. The motivation is that neural networks typically contain many more weights than nodes: between a layer with $N_{\ell-1}$ inputs and N_ℓ outputs, weight perturbation requires $N_{\ell-1}N_\ell$ perturbations, whereas node perturbation requires only N_ℓ . Node perturbation can therefore be viewed as a lower-dimensional perturbation method that preserves the same basic ingredients as weight perturbation: stochastic fluctuations, local synaptic information, and a global scalar learning signal.

To derive the standard node perturbation update, we use the same score-function logic as for weight perturbation, but apply it to perturbations of neural pre-activations rather than parameters. Thus, in this section, $\boldsymbol{\mu}$ denotes the deterministic vector of pre-activations to be perturbed, and $\boldsymbol{\theta}$ denotes the corresponding perturbed pre-activation vector:

$$\boldsymbol{\theta} \sim \mathcal{N}(\boldsymbol{\mu}, \sigma^2 I).$$

Equivalently, using the reparameterization

$$\boldsymbol{\theta} = \boldsymbol{\mu} + \sigma \boldsymbol{\xi}, \quad \boldsymbol{\xi} \sim \mathcal{N}(0, I),$$

node perturbation estimates how changes in the perturbed pre-activations affect the loss.

As in the weight perturbation case, the score-function estimator gives

$$\nabla_{\boldsymbol{\mu}} F(\boldsymbol{\mu}) = \mathbb{E} \left[L(\boldsymbol{\mu} + \sigma \boldsymbol{\xi}) \frac{\boldsymbol{\xi}}{\sigma} \right],$$

where $F(\boldsymbol{\mu}) = \mathbb{E}_{\boldsymbol{\theta} \sim p_{\boldsymbol{\mu}}} [L(\boldsymbol{\theta})]$ is the expected loss under node perturbations. For a single perturbation sample, and with a baseline b independent of the individual perturbation component ξ_j^ℓ , this gives the pre-activation-level estimator

$$\widehat{g}_{z_j^\ell} = (L(\boldsymbol{\mu} + \sigma \boldsymbol{\xi}) - b) \frac{\xi_j^\ell}{\sigma}.$$

Equivalently, if $\epsilon_j^\ell = \sigma \xi_j^\ell$, then

$$\widehat{g}_{z_j^\ell} = (L(\boldsymbol{\mu} + \boldsymbol{\epsilon}) - b) \frac{\epsilon_j^\ell}{\sigma^2}.$$

This estimator gives the gradient estimate with respect to the perturbed pre-activation z_j^ℓ . To update the network parameters, we must translate this pre-activation-level estimate into estimates for the incoming weights and bias.

Using the local derivatives from the pre-activation equation above,

$$\frac{\partial z_j^\ell}{\partial W_{ij}^\ell} = x_i^{\ell-1}, \quad \frac{\partial z_j^\ell}{\partial b_j^\ell} = 1,$$

the estimated weight gradient is

$$\widehat{g}_{W_{ij}^\ell} = \widehat{g}_{z_j^\ell} \frac{\partial z_j^\ell}{\partial W_{ij}^\ell} = (L(\boldsymbol{\mu} + \sigma \boldsymbol{\xi}) - b) \frac{\xi_j^\ell}{\sigma} x_i^{\ell-1},$$

and the estimated bias gradient is

$$\widehat{g}_{b_j^\ell} = \widehat{g}_{z_j^\ell} = (L(\boldsymbol{\mu} + \sigma \boldsymbol{\xi}) - b) \frac{\xi_j^\ell}{\sigma}.$$

Applying gradient descent gives the standard score-function-style node perturbation updates

$$\Delta W_{ij}^\ell = -\eta (L(\boldsymbol{\mu} + \sigma \boldsymbol{\xi}) - b) \frac{\xi_j^\ell}{\sigma} x_i^{\ell-1},$$

and

$$\Delta b_j^\ell = -\eta (L(\boldsymbol{\mu} + \sigma \boldsymbol{\xi}) - b) \frac{\xi_j^\ell}{\sigma}.$$

Thus, node perturbation has the same basic score-function structure as weight perturbation, but the perturbation is applied to pre-activations and then assigned to incoming weights using local presynaptic activity. Compared to WP, the NP update only adds the presynaptic activity term $x_i^{\ell-1}$, which is locally available to the incoming synapse. The update therefore fits naturally within the three-factor structure in Eq. (1): a local synaptic factor, a stochastic perturbation of neural activity, and a global scalar feedback signal. In this sense, node perturbation is also biologically plausible.

The update above represents the standard score-function-style form of node perturbation, but as for weight perturbation, several variants have been proposed in the literature. These methods differ mainly in where the perturbation is applied, how the perturbation is sampled or scaled, and how the scalar learning signal is formed. Table 1 summarizes the perturbation-based methods considered in this thesis. The most relevant alternatives are:

-
- **Finite-difference node perturbation.** Earlier methods, such as Madaline Rule III (Widrow and Lehr, 1990) and the neuron perturbation method of Flower and Jabri (Flower and Jabri, 1992), use the change in error caused by a node perturbation to approximate a local derivative. As in the finite-difference version of weight perturbation, the update is based on dividing the change in loss by the applied perturbation, rather than multiplying the scalar learning signal by the perturbation as in the score-function-style estimator.
 - **Noisy-baseline node perturbation.** Cho et al. (2011) and Hara et al. (2017) study variants where the baseline is not a noiseless forward pass, but the loss obtained from a second independent perturbation. This is motivated by the idea that biological neural activity is intrinsically noisy, making a noiseless baseline less biologically realistic.
 - **Perturbation-strength matching.** Werfel et al. (2005) scale the node perturbation variance in a linear setting so that node perturbation and weight perturbation produce comparable effective output perturbations. Since perturbing MN weights induces output noise whose variance grows with the number of inputs N , their node perturbation standard deviation is scaled by $\sqrt{N}$. Züge et al. (2023) use a related matching idea in temporally extended tasks, adjusting perturbation strength using an effective input dimension.

For compactness, Table 1 rewrites the notation from the different papers using the notation of this thesis. Perturbations are written as a scale multiplied by unit-scale noise, for example $\delta z = \sigma \xi$ with $\xi \sim \mathcal{N}(0, I)$, unless another scale is explicitly shown. The scalar learning signal is denoted by $\Delta \ell$. Let U_k denote the update direction, so that weights are updated as $W_k \leftarrow W_k - \eta_W U_k$. For decorrelated variants, $x_{k-1}^* = R_{k-1} x_{k-1}$, and the decorrelation matrix is updated using $R_k \leftarrow R_k - \eta_R (x_k^* (x_k^*)^\top - \text{diag}((x_k^*)^2)) R_k$. Method-specific constants that are absorbed into the learning rate in the original papers are kept at the same level of detail as in the source update rule.

Table 1: Comparison of perturbation-based update rules in the literature.

Reference	Method	Perturbed variable	Noise sampling	Update rule
Hiratani et al. (2022)	Node perturbation	Pre-activation	$\delta z_k = \sigma \xi_k, \xi_k \sim \mathcal{N}(0, I_k)$	$U_k = \frac{\Delta \ell}{\sigma} \xi_k x_{k-1}^\top$
Hiratani et al. (2022)	Weight-normalized node perturbation	Pre-activation	$\delta z_k = \sigma \xi_k, \xi_k \sim \mathcal{N}(0, I_k)$	$U_k = \frac{\Delta \ell}{\sigma} \xi_k x_{k-1}^\top, w_t^k \leftarrow \frac{\ w_t^k\ }{\ w_t^k - \eta_W u_t^k\ } (w_t^k - \eta_W u_t^k)$
Dalm et al. (2024)	Node perturbation	Pre-activation	$\delta z_k = \sigma \xi_k, \xi_k \sim \mathcal{N}(0, I_k)$	$U_k = \frac{\Delta \ell}{\sigma} \xi_k x_{k-1}^\top$
Dalm et al. (2024)	Iterative node perturbation	Pre-activation	$\delta z_k = \sigma \xi_k, \xi_k \sim \mathcal{N}(0, I_k)$	$U_k = \sqrt{N_k} \Delta \ell \frac{\xi_k}{\ \xi\ _2^2} x_{k-1}^\top$
Dalm et al. (2024)	Activity-based node perturbation	Pre-activation	$\delta z_k = \sigma \xi_k, \xi_k \sim \mathcal{N}(0, I_k)$	$U_k = \sqrt{N} \Delta \ell \frac{\delta \alpha_k}{\ \delta \alpha\ _2^2} x_{k-1}^\top$
Dalm et al. (2024)	Decorrelated node perturbation	Pre-activation	$\delta z_k = \sigma \xi_k, \xi_k \sim \mathcal{N}(0, I_k)$	$U_k = \frac{\Delta \ell}{\sigma} \xi_k (x_{k-1}^*)^\top$
Dalm et al. (2024)	Decorrelated iterative node perturbation	Pre-activation	$\delta z_k = \sigma \xi_k, \xi_k \sim \mathcal{N}(0, I_k)$	$U_k = \sqrt{N_k} \Delta \ell \frac{\xi_k}{\ \xi\ _2^2} (x_{k-1}^*)^\top$
Dalm et al. (2024)	Decorrelated activity-based node perturbation	Pre-activation	$\delta z_k = \sigma \xi_k, \xi_k \sim \mathcal{N}(0, I_k)$	$U_k = \sqrt{N} \Delta \ell \frac{\delta \alpha_k}{\ \delta \alpha\ _2^2} (x_{k-1}^*)^\top$
Flower and Jabri (1992)	Neuron perturbation	Pre-activation	$\delta z_{ij} = \sigma \xi_{ij}, \xi_{ij} \sim \text{Unif}\{-1, 1\}$	$U_{ij} = \frac{\Delta \ell}{\sigma \xi_{ij}} x_j$
Flower and Jabri (1992)	Weight perturbation	Weight	$\delta W_{ij} = \sigma \xi_{ij}, \xi_{ij} \sim \text{Unif}\{-1, 1\}$	$U_{ij} = \frac{\Delta \ell}{\sigma \xi_{ij}}$
Flower and Jabri (1992)	Summed weight neuron perturbation	Incoming weights	$\delta W_{ij} = \sigma \xi_{ij}, \xi_{ij} \sim \text{Unif}\{-1, 1\}$	$U_{ij} = \frac{\Delta \ell}{\sigma \xi_{ij}}, r_i = \sum_m \sigma \xi_{im} x_m$
Widrow and Lehr (1990)	Madaline Rule III	Pre-activation	$\delta z_i = \sigma \xi_i$; no distribution specified	$U_{ij} = \frac{\Delta \ell}{\sigma \xi_i} x_j$
Cauwenberghs (1993)	Stochastic error descent	Weights	$\delta \mathbf{p} = \sigma \xi, \mathbb{E}[\xi_i \xi_j] = \delta_{ij}$	$U = \Delta \ell \sigma \xi$
Werfel et al. (2005)	Node perturbation	Output	$\delta \mathbf{y} = \sqrt{N} \sigma \xi, \xi \sim \mathcal{N}(0, I_M)$	$U = \frac{\Delta \ell}{\sqrt{N} \sigma} \xi \mathbf{x}^\top$
Werfel et al. (2005)	Weight perturbation	Weight	$\delta \mathbf{W} = \sigma \xi, \xi \sim \mathcal{N}(0, I_{MN})$	$U = \frac{\Delta \ell}{\sigma} \xi$
Züge et al. (2023)	Weight perturbation	Weight	$\delta W_{ij} = \sigma_{\text{WP}} \xi_{ij}^{\text{WP}}, \xi_{ij}^{\text{WP}} \sim \mathcal{N}(0, 1)$	$U_{ij} = \frac{\Delta \ell}{\sigma_{\text{WP}}} \xi_{ij}^{\text{WP}}, \sigma_{\text{WP}} = \frac{\sigma_{\text{off}}}{\alpha \sqrt{N_{\text{eff}}}}$
Züge et al. (2023)	Node perturbation	Pre-activation	$\delta z_{it} = \sigma_{\text{eff}} \xi_{it}^{\text{NP}}, \xi_{it}^{\text{NP}} \sim \mathcal{N}(0, 1)$	$U_{ij} = \frac{\Delta \ell}{\sigma_{\text{eff}}} \sum_{t=1}^T \xi_{it}^{\text{NP}} x_{jt}$
Cho et al. (2011)	Node perturbation	Output	$\delta y_i = \sigma \xi_i, \xi_i \sim \mathcal{N}(0, 1)$	$U_{ij} = \Delta \ell \sigma \xi_i x_j$
Cho et al. (2011)	Noisy-baseline node perturbation	Output	$\delta y_i = \sigma \xi_i, \delta y_i' = \sigma \zeta_i, \xi_i, \zeta_i \sim \mathcal{N}(0, 1)$	$U_{ij} = \Delta \ell \sigma \xi_i x_j, \Delta \ell = L_\xi - L_\zeta$
Hara et al. (2017)	Single node perturbation	Output	$\delta y_k = \sigma \xi_k, \xi_k \sim \mathcal{N}(0, 1)$	$U_{jk} = \frac{\Delta \ell}{N \sigma} \xi_k x_j, \Delta \ell = L_\xi - L$
Hara et al. (2017)	Double node perturbation	Output	$\delta y_k = \sigma \xi_k, \delta y_k' = \sigma \zeta_k, \xi_k, \zeta_k \sim \mathcal{N}(0, 1)$	$U_{jk} = \frac{\Delta \ell}{N \sigma} \xi_k x_j, \Delta \ell = L_\xi - L_\zeta$

As for weight perturbation, the remainder of the thesis focuses on the score-function-style version of node perturbation rather than finite-difference methods. Earlier finite-difference variants, such as Madaline Rule III (Widrow and Lehr, 1990) and the neuron perturbation method of Flower and Jabri (1992), are therefore included as related work, but are not used in the later theoretical analysis or empirical evaluation. Similarly, noisy-baseline variants such as those studied by Cho et al. (2011) and Hara et al. (2017) are not treated as separate methods here, since the experiments in this thesis use a batch-mean baseline rather than a second noisy forward pass. The perturbation-strength matching idea is more directly relevant: Werfel et al. (2005) scale the node perturbation variance in a linear setting, and Züge et al. (2023) use a related scaling argument in temporally extended tasks. This motivates the fan-in-scaled NP variant considered later, and provides the closest existing comparison point for the input-scaled method developed in Chapter 3.

2.6 Existing comparisons of weight and node perturbation

Weight perturbation and node perturbation are closely related learning rules: both replace explicit gradient computation with random perturbations and a scalar feedback signal. This makes it natural to ask which method should be expected to learn more efficiently. The standard intuition is based on the dimensionality of the perturbation space. Weight perturbation injects noise into individual parameters, whereas node perturbation injects noise into neuron activities and then assigns the resulting learning signal to incoming weights using local presynaptic activity. Since neural networks typically contain many more weights than nodes, node perturbation is often expected to produce less noisy updates and faster learning. However, this dimensionality argument is only an intuition unless it is tied to a specific learning problem and estimator. The existing literature contains a small number of more direct comparisons between weight and node perturbation, which we review in this section.

An early comparison appears in the finite-difference setting studied by Flower and Jabri (Flower and Jabri, 1992). Their discussion presents standard weight perturbation as computationally more expensive than node perturbation, because weight perturbation estimates the effect of perturbing individual weights rather than nodes. This provides an early version of the idea that perturbing nodes can be more efficient than perturbing individual weights, although the comparison is made in a finite-difference framework rather than in the score-function-style setting considered in the rest of this thesis.

The most direct analytical comparison between WP and NP is given by Werfel et al. (Werfel et al., 2005). They analyze a linear perceptron with N inputs and M outputs, comparing direct gradient descent, node perturbation, and weight perturbation. In this setting, NP perturbs the M -dimensional output vector, whereas WP perturbs the full MN -dimensional weight matrix. Their learning-curve analysis shows that, when the scalar update parameter is chosen optimally, NP is slower than direct gradient descent by a factor equal to the number of output units M , while WP is slower than NP by an additional factor equal to the number of input units N . This provides a precise setting in which the lower perturbation dimension of NP leads to faster learning than WP. However, the result is derived for a simplified linear network with a specific input distribution, target function, and objective, and the authors explicitly note that the conclusions are not universally applicable.

Züge et al. (Züge et al., 2023) provide an important caveat to the usual dimensionality argument. Their analysis concerns temporally extended tasks, where a trial consists of multiple time steps before a scalar feedback signal is received. This is different from the standard feedforward setting considered above, where each input produces one forward pass and one loss value. In temporally extended tasks, the relevant perturbation dimension is more ambiguous: WP perturbs the weights once per trial, while NP may need to perturb neural activity across all time steps. As a result, the effective perturbation space explored by NP can grow with the duration of the trial, and may no longer be smaller than the space explored by WP. Their results therefore show that WP can perform similarly to, or better than, NP in some temporally extended tasks. This does not invalidate the standard argument that NP is often more efficient in feedforward settings, but it shows that the advantage is not universal and depends on the structure of the learning problem.

Later perturbation-learning papers reflect the same interpretation of the WP–NP relationship, often by referring back to the same core results discussed above. Lillicrap et al. (2020) describe weight perturbation as a simple scalar-feedback method, but state that it can be improved by perturbing neuron outputs rather than individual weights, referring to early node perturbation work such as Flower and Jabri (1992). Hiratani et al. (2022) make the dimensionality argument more explicit: they describe WP as noisier than NP in the vanilla setting, citing Werfel et al. (2005), while also noting that WP can be competitive with, or better than, NP when the feedback signal is sparse, citing Züge et al. (2023). Dalm et al. (2024) follow the same pattern, describing WP as searching a larger perturbation space because neural networks contain many more weights than nodes, while again citing the possibility that WP can outperform NP in special cases. Thus, later work appears to rely on the same basic picture: NP is often expected to be more efficient because it perturbs a lower-dimensional space, but this expectation is qualified by known exceptions where WP can be competitive or preferable.

2.7 Literature gap

The literature reviewed above provides a clear intuition for why node perturbation is often expected to be more efficient than weight perturbation: it perturbs a lower-dimensional space. This intuition is supported by analytical results in simplified linear settings and is commonly used in later work to motivate node perturbation over weight perturbation. However, the comparison remains incomplete. The strongest analytical result is derived for linear perceptrons, while more recent work shows that the usual dimensionality argument can fail in temporally extended tasks. Thus, although node perturbation is often treated as the more efficient default, the literature does not provide a general account of the relationship between weight perturbation and node perturbation in nonlinear multilayer networks.

This is the gap addressed in Chapter 3. Rather than asking only which method performs better empirically, we analyse how the two estimators are related theoretically in multilayer networks. As shown there, this analysis not only clarifies the relationship between weight-level and node-level perturbations, but also motivates a new variant of node perturbation that is designed to better match the perturbations induced by weight perturbation.

3 Original Theoretical Contribution

3.1 Overview of the contribution

This chapter addresses the second research objective of the thesis: to further develop the theory of perturbation-based learning. In particular, it addresses the literature gap identified in Chapter 2, where the relationship between WP and NP was found to be incomplete in nonlinear multilayer networks. The main theoretical result is a direct comparison between WP and a newly derived node perturbation variant, input-scaled node perturbation (IS-NP). We show that WP and IS-NP estimate the same perturbation-smoothed objective in expectation, but that IS-NP has variance smaller than or equal to WP for each coordinate. This gives a precise theoretical sense in which IS-NP is a variance-reduced node-level version of WP, and therefore clarifies the relationship between weight-level and node-level perturbations in nonlinear multilayer networks. At the same time, the derivation motivates IS-NP as a new perturbation rule: by scaling node noise according to the actual incoming activation norm, IS-NP is expected to be better matched to varying activation scales than standard NP and fan-in-scaled NP, although this comparison is later treated as a scaling argument rather than a formal variance theorem.

The line of reasoning behind the variance reduction proof is inspired by the local reparameterization trick of Kingma et al. (2015), but the setting and contribution are different. Kingma et al. show, in the context of stochastic variational inference in Bayesian neural networks, that sampling equivalent activation-level noise instead of weight-level noise can reduce gradient estimator variance. This thesis uses the same underlying idea in a different setting: perturbation-based learning, where

the question is not how to train a Bayesian neural network, but how WP and NP are related as scalar-feedback learning rules. We follow the same overall logic, but derive the corresponding result for perturbation-based learning. To the best of our knowledge, this connection has not previously been used to derive a node perturbation method from the noise induced by WP, nor to prove the corresponding variance reduction result for perturbation-based learning.

The chapter proceeds by first deriving the pre-activation noise induced by Gaussian WP, and then using this result to define IS-NP. We then show that WP and IS-NP have the same expected update because they estimate the same perturbation-smoothed objective, before proving that IS-NP has lower or equal coordinate-wise variance. This gives a clean theoretical comparison between IS-NP and WP. The chapter also discusses how IS-NP relates to standard NP and fan-in-scaled NP, but that comparison is based on scaling intuition rather than an equivalent variance theorem.

3.2 Pre-activation noise induced by WP

To compare WP and NP more directly, we first express the effect of WP at the same level where NP applies perturbations: the pre-activations. This allows us to characterize how independent weight perturbations propagate into neuron-level noise, and we can then compare this induced noise with the perturbations injected directly by NP. Recall from Section 2.4 that we modeled perturbations as a deterministic variable plus a scale times unit Gaussian noise. For neural networks, this corresponds to perturbing each weight and bias independently as

$$\mathbf{W}^{\ell'} = \mathbf{W}^{\ell} + \sigma \boldsymbol{\xi}_W^{\ell}, \quad \mathbf{b}^{\ell'} = \mathbf{b}^{\ell} + \sigma \boldsymbol{\xi}_b^{\ell}, \quad \xi_{W,ij}^{\ell}, \xi_{b,j}^{\ell} \sim \mathcal{N}(0, 1) \text{ i.i.d.},$$

where $\xi_{W,ij}^{\ell}$ denotes the (i, j) -entry of the unit-noise matrix $\boldsymbol{\xi}_W^{\ell}$, and $\xi_{b,j}^{\ell}$ denotes the unit-noise perturbation of bias b_j^{ℓ} .

The pre-activation at neuron j in layer ℓ is given by

$$z_j^{\ell} = \sum_i W_{ij}^{\ell} x_i^{\ell-1} + b_j^{\ell}.$$

After perturbation, this becomes

$$z_j^{\ell'} = \sum_i W_{ij}^{\ell'} x_i^{\ell-1'} + b_j^{\ell'} = \sum_i (W_{ij}^{\ell} + \sigma \xi_{W,ij}^{\ell}) x_i^{\ell-1'} + b_j^{\ell} + \sigma \xi_{b,j}^{\ell}.$$

This expression separates into a deterministic component and a stochastic component induced by the weight and bias perturbations:

$$z_j^{\ell'} = \sum_i W_{ij}^{\ell} x_i^{\ell-1'} + b_j^{\ell} + \sigma \sum_i \xi_{W,ij}^{\ell} x_i^{\ell-1'} + \sigma \xi_{b,j}^{\ell}.$$

Define the induced pre-activation noise as

$$\Delta z_j^{\ell} := \sigma \sum_i \xi_{W,ij}^{\ell} x_i^{\ell-1'} + \sigma \xi_{b,j}^{\ell}. \quad (13)$$

We can now write the full pre-activation in compact form as

$$z_j^{\ell'} = (\mathbf{W}^{\ell} \mathbf{x}^{\ell-1'})_j + b_j^{\ell} + \Delta z_j^{\ell},$$

Δz_j^{ℓ} is the effect of weight perturbation at the pre-activation level that we wanted to express. We now want to characterize the distribution of this noise term.

Condition on the incoming perturbed activations $\mathbf{x}^{\ell-1'}$. Under this conditioning, the coefficients $x_i^{\ell-1'}$ are fixed, while the weight- and bias-noise variables $\xi_{W,ij}^{\ell}$ and $\xi_{b,j}^{\ell}$ remain independent standard Gaussians. Therefore each weight-noise term $\sigma x_i^{\ell-1'} \xi_{W,ij}^{\ell}$ is Gaussian:

$$\sigma x_i^{\ell-1'} \xi_{W,ij}^{\ell} \mid \mathbf{x}^{\ell-1'} \sim \mathcal{N}(0, \sigma^2 (x_i^{\ell-1'})^2). \quad (14)$$

Since Δz_j^ℓ is a sum of independent Gaussian random variables, it is also Gaussian. Its conditional mean is

$$\mathbb{E}[\Delta z_j^\ell \mid \mathbf{x}^{\ell-1'}] = \sigma \sum_i x_i^{\ell-1'} \mathbb{E}[\xi_{W,ij}^\ell] = 0, \quad (15)$$

and its conditional variance is

$$\text{Var}(\Delta z_j^\ell \mid \mathbf{x}^{\ell-1'}) = \sigma^2 \sum_i (x_i^{\ell-1'})^2 \text{Var}(\xi_{W,ij}^\ell) + \sigma^2 \text{Var}(\xi_{b,j}^\ell) = \sigma^2 \left(1 + \|\mathbf{x}^{\ell-1'}\|^2\right). \quad (16)$$

Thus, the pre-activation noise induced by Gaussian weight perturbation satisfies

$$\Delta z_j^\ell \mid \mathbf{x}^{\ell-1'} \sim \mathcal{N}\left(0, \sigma^2 \left(1 + \|\mathbf{x}^{\ell-1'}\|^2\right)\right). \quad (17)$$

This shows that independent Gaussian perturbations at the weight and bias level do not produce arbitrary pre-activation noise. Instead, their combined effect at each neuron is itself Gaussian, with a variance determined by the norm of the incoming activation vector together with the bias contribution. Equivalently, the bias can be viewed as an additional input fixed at one. In other words, WP induces structured node-level noise: neurons receiving larger incoming activations are perturbed more strongly, while neurons receiving smaller incoming activations are perturbed less. This induced distribution is the key object we will use to compare WP with node perturbation directly.

3.3 Input-scaled node perturbation

The result above suggests a natural modification of standard node perturbation. Recall from Section 2.5 that standard NP injects Gaussian noise with the same perturbation scale at every node. That is, although the perturbation scale is a hyperparameter, it is fixed across neurons and does not depend on the magnitude of the incoming activation vector. This means that standard NP does not reproduce the pre-activation noise induced by WP: as shown in Section 3.2, Gaussian weight and bias perturbations induce node-level noise whose variance scales with $1 + \|\mathbf{x}^{\ell-1'}\|^2$.

One way to approximate this input-dependent scaling is fan-in-scaled NP. If the incoming activations are independent standard Gaussians, $x_i^{\ell-1} \sim \mathcal{N}(0, 1)$, then

$$\mathbb{E}\left[1 + \|\mathbf{x}^{\ell-1}\|^2\right] = \mathbb{E}\left[1 + \sum_{i=1}^{N_{\ell-1}} (x_i^{\ell-1})^2\right] = N_{\ell-1} + 1. \quad (18)$$

Under this assumption, the induced WP noise has expected variance $\sigma^2(N_{\ell-1} + 1)$ at each node when the bias is included. This is the same type of scaling argument used by Werfel et al. (2005), who choose the NP noise variance to be a factor N larger than the WP noise variance in order to make the two stochastic variants directly comparable. Fan-in-scaled NP therefore matches the WP-induced node noise under this distributional assumption, but only at the level of the expected squared input norm, with the bias treated as an additional constant input in this thesis.

We now remove this expectation-level approximation by matching the WP-induced pre-activation noise directly. This leads to the method proposed in this thesis, which we call input-scaled node perturbation (IS-NP). To the best of our knowledge, this specific perturbation rule has not been derived previously; in particular, it does not appear among the perturbation-learning methods reviewed in Table 1. For each neuron j in layer ℓ , we sample

$$\Delta z_j^\ell = \sigma \sqrt{1 + \|\mathbf{x}^{\ell-1'}\|^2} \xi_{z,j}^\ell, \quad \xi_{z,j}^\ell \sim \mathcal{N}(0, 1). \quad (19)$$

This gives the same conditional distribution as the WP-induced pre-activation noise derived in Section 3.2:

$$\Delta z_j^\ell \mid \mathbf{x}^{\ell-1'} \sim \mathcal{N}\left(0, \sigma^2 \left(1 + \|\mathbf{x}^{\ell-1'}\|^2\right)\right). \quad (20)$$

We now derive the corresponding update rule. Since the perturbation standard deviation is $\sigma\sqrt{1 + \|\mathbf{x}^{\ell-1'}\|^2}$, the score-function term for the pre-activation perturbation is

$$\frac{\Delta z_j^\ell}{\sigma^2 (1 + \|\mathbf{x}^{\ell-1'}\|^2)} = \frac{\xi_{z,j}^\ell}{\sigma\sqrt{1 + \|\mathbf{x}^{\ell-1'}\|^2}}. \quad (21)$$

Let $\Delta\ell = L_{\text{pert}} - b$ denote the scalar learning signal. The resulting gradient estimate with respect to the pre-activation is therefore

$$\hat{g}_{z_j^\ell} = \Delta\ell \frac{\xi_{z,j}^\ell}{\sigma\sqrt{1 + \|\mathbf{x}^{\ell-1'}\|^2}}. \quad (22)$$

Using the local derivatives

$$\frac{\partial z_j^\ell}{\partial W_{ij}^\ell} = x_i^{\ell-1'}, \quad \frac{\partial z_j^\ell}{\partial b_j^\ell} = 1, \quad (23)$$

we obtain the corresponding weight-gradient estimator

$$\hat{g}_{ij,\ell}^{\text{IS-NP}} = \Delta\ell \frac{\xi_{z,j}^\ell x_i^{\ell-1'}}{\sigma\sqrt{1 + \|\mathbf{x}^{\ell-1'}\|^2}}. \quad (24)$$

The corresponding bias-gradient estimator is

$$\hat{g}_{b_j^\ell}^{\text{IS-NP}} = \Delta\ell \frac{\xi_{z,j}^\ell}{\sigma\sqrt{1 + \|\mathbf{x}^{\ell-1'}\|^2}}. \quad (25)$$

Applying gradient descent gives the update rules

$$\Delta W_{ij}^\ell = -\eta \Delta\ell \frac{\xi_{z,j}^\ell x_i^{\ell-1'}}{\sigma\sqrt{1 + \|\mathbf{x}^{\ell-1'}\|^2}}. \quad (26)$$

and

$$\Delta b_j^\ell = -\eta \Delta\ell \frac{\xi_{z,j}^\ell}{\sigma\sqrt{1 + \|\mathbf{x}^{\ell-1'}\|^2}}. \quad (27)$$

Thus, unlike standard NP, the perturbation scale is not fixed across neurons. Unlike fan-in-scaled NP, it is also not determined only by the number of incoming units. Instead, IS-NP scales the perturbation by the actual norm of the incoming activation vector for each layer and input example. This does not by itself imply that IS-NP is preferable to other NP variants in practice, but it makes IS-NP directly comparable to WP because it samples the same pre-activation noise distribution induced by Gaussian weight perturbations.

In terms of biological plausibility, IS-NP remains close to the three-factor learning picture. Compared to standard NP, it also uses the norm of the full incoming activation vector together with the bias contribution, $\sqrt{1 + \|\mathbf{x}^{\ell-1'}\|^2}$, rather than only the activity of the single presynaptic neuron connected to a given synapse. This goes beyond the simplest form of Eq. (1), but the added information is still based on presynaptic activity arriving at the postsynaptic neuron, together with the node perturbation and the global scalar feedback signal. IS-NP therefore remains well aligned with the general ingredients of three-factor learning, while assuming an additional neuron-level scaling term compared to standard NP.

3.4 Expected equivalence between WP and IS-NP

Recall from Section 2.4 that WP does not optimize the deterministic loss directly, but the perturbation-smoothed objective

$$F(\theta) := \mathbb{E}_{\theta' \sim p_\theta} [L(\theta')], \quad (28)$$

where θ denotes the trainable network parameters, including both weights and biases. For WP, the perturbed parameters can be written as $\theta' = \theta + \Delta\theta$, so the objective is

$$F_{\text{WP}}(\theta) = \mathbb{E}_{\Delta\theta} [L(\text{forward}(\theta + \Delta\theta))]. \quad (29)$$

Although this expectation is defined over parameter perturbations, the loss does not depend on $\Delta\theta$ directly. It depends on $\Delta\theta$ only through the perturbed forward pass that these parameter perturbations produce.

Let

$$\Delta\mathbf{Z} = (\Delta\mathbf{z}^1, \dots, \Delta\mathbf{z}^L) \quad (30)$$

denote the full induced pre-activation noise field. Since $\Delta\mathbf{Z}$ determines the perturbed forward pass, WP can equivalently be viewed as inducing a distribution over $\Delta\mathbf{Z}$. If this induced distribution is denoted by $p_{\text{WP}}(\Delta\mathbf{Z})$, then the WP objective can be written as

$$F_{\text{WP}}(\theta) = \mathbb{E}_{\Delta\mathbf{Z} \sim p_{\text{WP}}} [L(\text{forward}(\theta; \Delta\mathbf{Z}))]. \quad (31)$$

Equivalently,

$$F_{\text{WP}}(\theta) = \int L(\text{forward}(\theta; \Delta\mathbf{Z})) p_{\text{WP}}(\Delta\mathbf{Z}) d\Delta\mathbf{Z}. \quad (32)$$

This expression makes clear that the objective is an average over possible perturbed forward passes: each possible pre-activation noise field contributes its loss value, weighted by the probability that WP induces that noise field.

From Section 3.2, WP induces the conditional distribution

$$\Delta z_j^\ell \mid \mathbf{x}^{\ell-1'} \sim \mathcal{N}\left(0, \sigma^2 \left(1 + \|\mathbf{x}^{\ell-1'}\|^2\right)\right). \quad (33)$$

IS-NP is defined to sample directly from this same conditional distribution. Therefore, for each layer,

$$p_{\text{IS-NP}}(\Delta\mathbf{z}^\ell \mid \mathbf{x}^{\ell-1'}) = p_{\text{WP}}(\Delta\mathbf{z}^\ell \mid \mathbf{x}^{\ell-1'}). \quad (34)$$

The forward recursion is deterministic once $\Delta\mathbf{z}^\ell$ is sampled:

$$\mathbf{z}^{\ell'} = \mathbf{W}^\ell \mathbf{x}^{\ell-1'} + \mathbf{b}^\ell + \Delta\mathbf{z}^\ell, \quad \mathbf{x}^{\ell'} = \phi(\mathbf{z}^{\ell'}). \quad (35)$$

Hence the equality of the layer-wise conditional distributions propagates through the network, giving the same distribution over the full pre-activation noise field:

$$p_{\text{IS-NP}}(\Delta\mathbf{Z}) = p_{\text{WP}}(\Delta\mathbf{Z}). \quad (36)$$

The IS-NP objective can therefore be written as

$$F_{\text{IS-NP}}(\theta) = \int L(\text{forward}(\theta; \Delta\mathbf{Z})) p_{\text{IS-NP}}(\Delta\mathbf{Z}) d\Delta\mathbf{Z} \quad (37)$$

$$= \int L(\text{forward}(\theta; \Delta\mathbf{Z})) p_{\text{WP}}(\Delta\mathbf{Z}) d\Delta\mathbf{Z} \quad (38)$$

$$= F_{\text{WP}}(\theta). \quad (39)$$

Thus, WP and IS-NP define the same perturbation-smoothed objective. For every possible $\Delta\mathbf{Z}$, the two methods assign the same probability to the corresponding perturbed forward pass, and the loss value produced by that forward pass is the same.

Since both WP and IS-NP are unbiased score-function estimators of the gradient of this same objective, their expected gradient estimates are equal:

$$\mathbb{E}[\hat{g}^{\text{WP}}] = \nabla F(\theta) = \mathbb{E}[\hat{g}^{\text{IS-NP}}]. \quad (40)$$

In other words, WP and IS-NP have the same expected update direction. The difference between them is therefore not the objective they estimate, but the variance of their gradient estimators.

The same argument does not generally apply to standard NP or fan-in-scaled NP. Standard NP samples fixed-variance pre-activation noise,

$$\Delta z_j^\ell \sim \mathcal{N}(0, \sigma^2), \quad (41)$$

whereas WP induces

$$\Delta z_j^\ell \mid \mathbf{x}^{\ell-1'} \sim \mathcal{N}\left(0, \sigma^2 \left(1 + \|\mathbf{x}^{\ell-1'}\|^2\right)\right). \quad (42)$$

These distributions are equal only in the special case where

$$\|\mathbf{x}^{\ell-1'}\|^2 = 0. \quad (43)$$

Similarly, fan-in-scaled NP samples noise with variance proportional to the number of incoming units,

$$\Delta z_j^\ell \sim \mathcal{N}\left(0, \sigma^2(N_{\ell-1} + 1)\right). \quad (44)$$

This matches the WP-induced distribution only when

$$\|\mathbf{x}^{\ell-1'}\|^2 = N_{\ell-1}. \quad (45)$$

Thus, standard NP and fan-in-scaled NP generally induce different distributions over pre-activation perturbations than WP and IS-NP. They may match WP under special normalization assumptions, but not in the general case where the incoming activation norm varies across layers, samples, and training time.

This distinction is important for the variance analysis that follows. Since WP and IS-NP estimate the gradient of the same perturbation-smoothed objective, comparing their variance is a clean estimator-level comparison: both estimators have the same expectation, and the only question is how much they vary around it. The same is not generally true for standard NP or fan-in-scaled NP, since these methods may target slightly different smoothed objectives. IS-NP therefore provides something that the existing NP variants do not: a node-level perturbation rule that can be compared directly with WP at the level of estimator variance.

3.5 Variance reduction theorem

The previous section showed that IS-NP and WP are matched in expectation. We now show that they differ in variance. More specifically, IS-NP can be viewed as the conditional expectation of WP given the induced pre-activation noise field. This means that IS-NP removes weight-noise variability that affects the WP update but does not affect the actual perturbed forward pass. The result is a direct variance reduction by the law of total variance.

For a single weight coordinate W_{ij}^ℓ , the WP estimator can be written as

$$\hat{g}_{ij,\ell}^{\text{WP}} = \Delta \ell \frac{\xi_{W,ij}^\ell}{\sigma}. \quad (46)$$

The corresponding bias-coordinate estimator is

$$\hat{g}_{b_j^\ell}^{\text{WP}} = \Delta \ell \frac{\xi_{b,j}^\ell}{\sigma}. \quad (47)$$

Proposition 1. *For Gaussian weight and bias perturbations and the corresponding input-scaled node perturbation estimator defined in Eqs. (24) and (25), the coordinate-wise variance of IS-NP is no larger than that of WP. For each weight coordinate,*

$$\text{Var}(\hat{g}_{ij,\ell}^{\text{IS-NP}}) \leq \text{Var}(\hat{g}_{ij,\ell}^{\text{WP}}).$$

The same inequality holds for each bias coordinate b_j^ℓ .

Proof. We consider the weight perturbation estimator in (46), and analyze its variance by conditioning on the induced pre-activation noise. Recall from (13) that Δz_j^ℓ is a linear combination of the weight perturbations.

Condition on the full induced noise field $\Delta \mathbf{Z} = (\Delta \mathbf{z}^1, \dots, \Delta \mathbf{z}^L)$. Since the perturbed forward pass satisfies

$$\mathbf{z}^{\ell'} = \mathbf{W}^\ell \mathbf{x}^{\ell-1'} + \mathbf{b}^\ell + \Delta \mathbf{z}^\ell,$$

the entire perturbed trajectory $(\mathbf{x}^{1'}, \dots, \mathbf{x}^{L'})$ is uniquely determined by $\Delta \mathbf{Z}$. Hence both the scalar learning signal $\Delta \ell(\Delta \mathbf{Z})$ and all activations $\mathbf{x}^{\ell'}$ are deterministic given $\Delta \mathbf{Z}$.

Now fix (i, j, ℓ) . Given $\Delta \mathbf{Z}$, the incoming activation vector $\mathbf{x}^{\ell-1'}$ is fixed, and the only component of the induced noise field that contains information about $\xi_{W,ij}^\ell$ is Δz_j^ℓ . From (13), Δz_j^ℓ is a linear combination of Gaussian weight and bias perturbations. Hence, $\xi_{W,ij}^\ell$ and Δz_j^ℓ are jointly Gaussian. Since both variables have mean zero, the conditional expectation is linear and given by

$$\mathbb{E}[\xi_{W,ij}^\ell \mid \Delta \mathbf{Z}] = \frac{\text{Cov}(\xi_{W,ij}^\ell, \Delta z_j^\ell)}{\text{Var}(\Delta z_j^\ell)} \Delta z_j^\ell. \quad (48)$$

The covariance term is

$$\text{Cov}(\xi_{W,ij}^\ell, \Delta z_j^\ell) = \text{Cov}\left(\xi_{W,ij}^\ell, \sigma \sum_k \xi_{W,kj}^\ell x_k^{\ell-1'} + \sigma \xi_{b,j}^\ell\right) \quad (49)$$

$$= \sigma \sum_k x_k^{\ell-1'} \text{Cov}(\xi_{W,ij}^\ell, \xi_{W,kj}^\ell) \quad (50)$$

$$= \sigma x_i^{\ell-1'}, \quad (51)$$

where the last equality follows from the independence and unit variance of the Gaussian weight perturbations. Similarly,

$$\text{Var}(\Delta z_j^\ell) = \sigma^2 \left(1 + \|\mathbf{x}^{\ell-1'}\|^2\right). \quad (52)$$

Substituting these expressions gives

$$\mathbb{E}[\xi_{W,ij}^\ell \mid \Delta \mathbf{Z}] = \frac{x_i^{\ell-1'}}{\sigma \left(1 + \|\mathbf{x}^{\ell-1'}\|^2\right)} \Delta z_j^\ell. \quad (53)$$

Therefore,

$$\mathbb{E}[\hat{g}_{ij,\ell}^{\text{WP}} \mid \Delta \mathbf{Z}] = \frac{\Delta \ell(\Delta \mathbf{Z})}{\sigma} \mathbb{E}[\xi_{W,ij}^\ell \mid \Delta \mathbf{Z}] = \Delta \ell(\Delta \mathbf{Z}) \frac{x_i^{\ell-1'} \Delta z_j^\ell}{\sigma^2 \left(1 + \|\mathbf{x}^{\ell-1'}\|^2\right)}. \quad (54)$$

Recall from (24) that the IS-NP estimator is

$$\hat{g}_{ij,\ell}^{\text{IS-NP}} = \Delta \ell \frac{\xi_{z,j}^\ell x_i^{\ell-1'}}{\sigma \sqrt{1 + \|\mathbf{x}^{\ell-1'}\|^2}}. \quad (55)$$

Since the IS-NP perturbation satisfies

$$\Delta z_j^\ell = \sigma \sqrt{1 + \|\mathbf{x}^{\ell-1'}\|^2} \xi_{z,j}^\ell, \quad (56)$$

the same estimator can equivalently be written as

$$\hat{g}_{ij,\ell}^{\text{IS-NP}} = \Delta \ell \frac{x_i^{\ell-1'} \Delta z_j^\ell}{\sigma^2 \left(1 + \|\mathbf{x}^{\ell-1'}\|^2\right)}. \quad (57)$$

Hence,

$$\mathbb{E}[\hat{g}_{ij,\ell}^{\text{WP}} \mid \Delta \mathbf{Z}] = \hat{g}_{ij,\ell}^{\text{IS-NP}}. \quad (58)$$

The law of total variance states that for random variables Y and X ,

$$\text{Var}(Y) = \text{Var}(\mathbb{E}[Y | X]) + \mathbb{E}[\text{Var}(Y | X)].$$

Applying this identity with $Y = \hat{g}_{ij,\ell}^{\text{WP}}$ and $X = \Delta \mathbf{Z}$,

$$\text{Var}(\hat{g}_{ij,\ell}^{\text{WP}}) = \text{Var}(\mathbb{E}[\hat{g}_{ij,\ell}^{\text{WP}} | \Delta \mathbf{Z}]) + \mathbb{E}[\text{Var}(\hat{g}_{ij,\ell}^{\text{WP}} | \Delta \mathbf{Z})].$$

Using the identity above,

$$\text{Var}(\hat{g}_{ij,\ell}^{\text{WP}}) = \text{Var}(\hat{g}_{ij,\ell}^{\text{IS-NP}}) + \mathbb{E}[\text{Var}(\hat{g}_{ij,\ell}^{\text{WP}} | \Delta \mathbf{Z})].$$

Since the second term is nonnegative, it follows that

$$\text{Var}(\hat{g}_{ij,\ell}^{\text{IS-NP}}) \leq \text{Var}(\hat{g}_{ij,\ell}^{\text{WP}}).$$

The proof for the bias coordinate is identical, with the constant bias input taking the place of $x_i^{\ell-1'}$. In that case,

$$\text{Cov}(\xi_{b,j}^\ell, \Delta z_j^\ell) = \sigma,$$

which gives

$$\mathbb{E}[\xi_{b,j}^\ell | \Delta \mathbf{Z}] = \frac{\Delta z_j^\ell}{\sigma (1 + \|\mathbf{x}^{\ell-1'}\|^2)}.$$

Substituting this into the WP bias estimator in Eq. (47) gives the IS-NP bias estimator in Eq. (25), so the same law-of-total-variance argument applies. $\square$

The intuition behind this result is simple. Once the induced pre-activation noise field $\Delta \mathbf{Z}$ is fixed, the perturbed forward pass is fixed. This means that the activations, the loss difference $\Delta \ell$, and the IS-NP update are all determined. In WP, however, the update still depends on the individual weight perturbations that produced the same $\Delta \mathbf{Z}$. These remaining weight-level perturbations no longer affect the forward pass or the loss signal; they only change the WP update itself. They therefore add variability that does not help learning. From the perspective of estimating the same learning signal, this variability is simply extra noise.

The variance theorem completes the comparison introduced at the start of the chapter. It shows that the difference between weight-level and node-level perturbation is not only a matter of perturbation dimensionality: when the node perturbation is chosen to match the noise induced by WP, the resulting estimator has the same expectation as WP but lower or equal coordinate-wise variance. This means that the weight-level estimator contains variability that is irrelevant once the induced pre-activation noise is fixed. In this precise sense, WP can be viewed as a higher-variance way of sampling a learning signal that can also be sampled at the node level. The result therefore clarifies the relationship between weight-level and node-level perturbations in nonlinear multilayer networks, while also identifying IS-NP as the node-level rule for which this comparison is exact.

3.6 Why IS-NP may improve over existing NP variants

The variance theorem above compares IS-NP directly with WP, but it does not establish how IS-NP compares to other node perturbation variants. As shown in section 3.4, NP and fan-in-scaled NP use different perturbation distributions, and therefore generally correspond to perturbation-smoothed objectives that are similar, but not identical, to those of WP and IS-NP. For this reason, we do not derive an analytical variance comparison between IS-NP and these methods. Instead, we use the scaling arguments developed above to reason about which perturbation rule is likely to be better behaved in practice, before testing this empirically in Chapter 5.

Standard NP uses the same perturbation scale at every node, regardless of the incoming activation norm. This can lead to poorly scaled updates. If $\|\mathbf{x}^{\ell-1}\|$ is large, a fixed pre-activation perturbation may have only a small relative effect on the network output, while the resulting weight update

can still become large because it is multiplied by the incoming activation. Conversely, if $\|\mathbf{x}^{\ell-1}\|$ is small, the same fixed perturbation may create a large relative change in the pre-activation, while the corresponding weight update is small. In both cases, the perturbation scale is disconnected from the activation scale that determines the weight update. This mismatch is expected to be most problematic in networks where activation norms vary substantially across layers, samples, or training time, for example when layer widths differ or activations are not explicitly normalized. In such cases, a fixed perturbation scale will sometimes be too large and sometimes too small, making standard NP less consistently scaled than methods that account for the incoming activation norm.

Fan-in-scaled NP partially addresses this issue by accounting for the number of incoming units. If the incoming activations are independent, zero-mean, and unit-variance, and the bias is treated as an additional constant input, then

$$\mathbb{E}\left[1 + \|\mathbf{x}^{\ell-1}\|^2\right] = \mathbb{E}\left[1 + \sum_{i=1}^{N_{\ell-1}} (x_i^{\ell-1})^2\right] = 1 + \sum_{i=1}^{N_{\ell-1}} \mathbb{E}[(x_i^{\ell-1})^2] = N_{\ell-1} + 1. \quad (59)$$

Thus, under these assumptions, the expected squared norm of the incoming activation vector, including the bias contribution, grows with the fan-in. This is the same scaling argument used by Werfel et al. (2005), who choose the NP perturbation variance to be a factor N larger than the WP perturbation variance so that the two stochastic variants are directly comparable in their linear setting. If the incoming activations were explicitly normalized so that their squared norm matched the layer width, this correction would be exact up to the details of the normalization. However, this would introduce an additional architectural mechanism that is not part of the perturbation-learning methods reviewed here. Some reviewed work uses related stabilization or preprocessing techniques, such as the weight-normalized NP variant of Hiratani et al. (2022) and the input-decorrelated NP variants of Dalm et al. (2024), but these do not correspond to layer-wise activation normalization that fixes $1 + \|\mathbf{x}^{\ell-1}\|^2$ in each forward pass. Moreover, common artificial normalization methods such as layer normalization typically include learnable scale and shift parameters, so they do not generally imply a fixed activation norm unless constrained to do so. Relying on such normalization would therefore make fan-in scaling depend on an extra assumption, and one whose biological interpretation is not established in the perturbation-learning literature considered here. In the setting studied in this thesis, activation norms are therefore allowed to vary across layers, samples, and training time. Fan-in scaling can therefore improve on standard NP by correcting for the typical size of a layer, but it still does not account for the actual incoming activation norm in a given forward pass.

IS-NP avoids this average correction by using the actual incoming activation norm, together with the bias contribution, in the current forward pass. Its perturbation scale is therefore adapted to the layer, input example, and current network state. This does not prove that IS-NP must outperform standard NP or fan-in-scaled NP in every practical setting, since other factors may also influence learning performance. However, it gives a clear scaling-based reason to expect more consistent behaviour when activation norms vary across layers, samples, or training time. Whether this advantage translates into better empirical performance is therefore a question to be tested experimentally.

3.7 Summary of theoretical contribution

This chapter addressed the second research objective of the thesis: to further develop the theory of perturbation-based learning. Building on the gap identified in Chapter 2, we analysed the relationship between WP and NP at the estimator level in nonlinear multilayer networks. This led to the derivation of input-scaled node perturbation (IS-NP), a node perturbation rule that samples the pre-activation noise induced by Gaussian WP directly. We then showed that WP and IS-NP estimate the same perturbation-smoothed objective in expectation, and proved that IS-NP has variance smaller than or equal to WP for each coordinate. Finally, we compared IS-NP conceptually with standard NP and fan-in-scaled NP. This gave scaling-based reasons to expect IS-NP to behave more consistently, although this comparison was not a formal variance proof.

Chapters 2 and 3 therefore establish the theoretical foundation for the thesis. Chapter 2 addressed

the first research objective by reviewing and clarifying the existing theory of perturbation-based learning, while Chapter 3 addressed the second objective. What remains is to address the third research objective: to evaluate how WP, standard NP, fan-in-scaled NP, and IS-NP perform in practice. The next chapter therefore turns from theory to experiments, where these four learning rules are compared empirically across the selected benchmark tasks.

4 Method

4.1 Empirical evaluation strategy

The goal of this chapter is to address the third research objective of the thesis: to evaluate perturbation-based learning methods empirically. This is necessary because neural network optimization problems are generally high-dimensional and non-convex, and the behaviour of a learning rule depends on more than its expected update or estimator variance in isolation. Although the theoretical analysis in Chapter 3 gives useful predictions about the relationship between WP and IS-NP, and motivates why IS-NP may be better scaled than vanilla NP and fan-in-scaled NP, these results do not by themselves determine practical learning performance. A variance reduction result can be mathematically correct while still having limited practical effect if other factors dominate the optimization dynamics. Similarly, the argument that IS-NP is better scaled than vanilla or fan-in-scaled NP is theoretically motivated, but it is not a proof that IS-NP will train better in all tasks. To understand their practical usefulness, the methods must therefore be tested empirically. The evaluation is divided into two related parts.

The first part asks how well perturbation-based learning methods work in practice as alternatives to backpropagation. Backpropagation is included as the standard reference method because it computes the exact gradient that the perturbation methods attempt to estimate. The goal is not to show that perturbation methods outperform backpropagation, but to understand how close they can get, how difficult a task they can solve, and where their performance begins to break down. Therefore, worse performance than backpropagation should not be interpreted as a failure of the empirical evaluation. It is an expected and informative part of the comparison, because it shows the practical cost of replacing exact gradient computation with stochastic perturbation-based estimates. This is important in relation to the motivation of Chapter 1: if backpropagation is difficult or impossible to use in biologically inspired architectures or other constrained settings, it is useful to know how capable perturbation-based learning methods are as substitutes.

The second part asks whether the proposed input-scaled node perturbation method improves over existing perturbation methods in the way suggested by the theoretical contribution in Chapter 3. In particular, the experiments compare IS-NP with WP, vanilla NP, and fan-in-scaled NP. The comparison with WP tests whether the variance reduction theorem has practical consequences for learning performance and estimator quality. The comparison with the other NP variants tests whether scaling perturbations by the actual incoming activation norm improves over fixed or layer-width-based perturbation scales.

To answer these questions, the methods are evaluated on four supervised learning problems of increasing complexity: synthetic sinus regression, California housing regression, MNIST classification, and CIFAR-10 classification. This progression is chosen so that the experiments can show not only whether a method works, but also how its performance changes as the learning problem becomes more difficult. Simple tasks reveal whether the methods can learn under favourable conditions, while harder tasks make the differences between learning rules more visible. The purpose of including difficult tasks is both to clearly reveal differences between methods and to identify where the perturbation methods become inefficient, unstable, or unable to match the reference performance of backpropagation. The natural next question is therefore: what does it mean for a learning method to work well?

4.2 Learning problems and datasets

The empirical evaluation uses four supervised learning problems of increasing complexity. Three of these are standard datasets commonly used in machine learning experiments: California housing, MNIST, and CIFAR-10. The fourth is a synthetic sinus regression task generated specifically to provide a simple and controlled nonlinear regression problem. Together, these tasks span low-dimensional synthetic regression, real tabular regression, simple image classification, and more difficult image classification. The purpose is not to maximize benchmark performance on any one dataset, but to compare how the learning rules behave as the problem becomes more challenging.

The preprocessing is kept simple and standard. The imported datasets are used in their usual form, with only the transformations needed to make them suitable for the fully connected models used in the experiments. This includes standardization for the tabular regression task and pixel scaling, mean-centering, flattening, and one-hot label encoding for the image classification tasks. The following sections describe each learning problem before summarizing the dataset properties in Table 2.

4.2.1 Synthetic sinus regression

The sinus task is a synthetic regression problem designed to provide a controlled nonlinear learning problem. The input is an eight-dimensional vector, and the target is generated from a smooth nonlinear function containing sinusoidal terms, quadratic terms, and a small interaction term. This makes the task more interesting than a linear regression problem while still being simple enough that all learning methods should be expected to make progress.

The dataset consists of 512 training examples and 512 test examples. The task is a scalar regression problem, so each input vector is mapped to one real-valued output. The training inputs are sampled uniformly from $[-1, 1]^8$. Training targets are corrupted with additive Gaussian noise with standard deviation 0.1, while the test targets are generated from the noiseless target function.

No external preprocessing is applied to the sinus inputs in the main experiment. The model receives the sampled input vector directly, so the input values remain in the range $[-1, 1]$.

4.2.2 California housing regression

The California housing task is a real tabular regression problem. The goal is to predict median house value from housing and geographical features describing districts in California. Compared with the sinus task, this is a less controlled problem because the data come from a real dataset rather than a generated function, but it remains low-dimensional and is therefore a useful intermediate regression task.

The dataset contains 20,640 examples. It is split into 16,512 training examples and 4,128 test examples, corresponding to an 80/20 train-test split. Each input has eight features, and the output is one real-valued target, so the task is scalar regression.

The input features are standardized using the mean and standard deviation computed from the training split. The target values are standardized in the same way. The test set is transformed using the training-split statistics, so no information from the test set is used during preprocessing.

4.2.3 MNIST classification

MNIST is a handwritten digit classification task. Each example is a grayscale image of a digit from 0 to 9, and the goal is to assign the image to the correct digit class. The task is substantially higher-dimensional than the two regression tasks, but the images are simple and centered, making MNIST a standard first benchmark for image classification methods.

The full MNIST dataset contains 60,000 training images and 10,000 test images. In this thesis,

a subset of 20,000 training examples and 5,000 test examples is used. Because the results were already clear when using 20,000 training examples, this subset size was kept in order to reduce computational cost. Each image has size 28×28 , which gives 784 input values after flattening. The output is a 10-dimensional vector corresponding to the ten digit classes, and the task is classification.

Pixel values are scaled to $[0, 1]$, mean-centered using the scalar mean of the training subset, and flattened into vectors. The class labels are converted to one-hot vectors for training.

4.2.4 CIFAR-10 classification

CIFAR-10 is an image classification task with ten object classes. Each example is a small colour image, and the goal is to assign the image to the correct class. Compared with MNIST, CIFAR-10 is substantially more difficult because the images contain colour, background variation, and more complex visual structure. It is therefore included as the hardest task in the empirical evaluation.

The full CIFAR-10 dataset contains 50,000 training images and 10,000 test images. In this thesis, a subset of 20,000 training examples and 5,000 test examples is used. As for MNIST, because the results were already clear when using 20,000 training examples, this subset size was kept in order to reduce computational cost. Each image has size 32×32 with three colour channels, giving 3072 input values after flattening. The output is a 10-dimensional vector corresponding to the ten classes, and the task is classification.

Pixel values are scaled to $[0, 1]$, mean-centered using the scalar mean of the training subset, rearranged to channel-first format, and flattened into vectors. The class labels are converted to one-hot vectors for training.

4.2.5 Scaled-input diagnostics

In addition to the four main learning problems, a scaled-input diagnostic experiment is performed on the sinus task. In the main sinus experiment, inputs lie in $[-1, 1]^8$. In the scaled-input version, the same underlying target function is used, but the model input is multiplied by 5, giving inputs in $[-5, 5]^8$. The target values are not changed. This isolates the effect of increasing the input norm while keeping the learning problem itself otherwise fixed.

The purpose of this diagnostic is to test how sensitive the perturbation estimators are to changes in activation scale. Vanilla NP uses a fixed node perturbation scale, while fan-in NP scales only by layer width. Both methods therefore ignore the actual norm of the incoming activation vector. IS-NP, in contrast, scales the perturbation using the incoming activation norm, and WP naturally produces pre-activation perturbations whose scale depends on the input. The scaled sinus setting is artificial in the sense that one would normally avoid unnecessary input rescaling, but it provides a clean way to test what happens when the scale assumptions behind fixed or layer-width-based node perturbation are violated.

This is especially useful because the sinus architecture uses equal hidden-layer widths, so the diagnostic is not confounded by different layer sizes. The experiment therefore focuses directly on the effect of input scale. The same kind of scale mismatch could also occur inside deeper networks if activation norms vary strongly across layers or samples, but the sinus diagnostic makes the effect visible in a simple controlled setting.

Table 2: Learning problems used in the empirical evaluation.

Dataset	Task type	Input $\rightarrow$ output	Train size	Test size	Score	Source	Difficulty
Synthetic sinus	Regression	$8 \rightarrow 1$	512	512	R^2	Generated in the experiment	1
California housing	Regression	$8 \rightarrow 1$	16,512	4,128	R^2	<code>sklearn.datasets.fetch_california_housing</code>	2
MNIST	Classification	$784 \rightarrow 10$	20,000	5,000	Accuracy	<code>torchvision.datasets.MNIST</code>	3
CIFAR-10	Classification	$3072 \rightarrow 10$	20,000	5,000	Accuracy	<code>torchvision.datasets.CIFAR10</code>	4

4.3 Model architecture and initialization

All experiments use fully connected multilayer perceptrons (MLPs). This choice is made to keep the empirical evaluation aligned with the theoretical setting considered in the thesis and to isolate differences between learning rules rather than differences between architectures. For image classification tasks such as MNIST and CIFAR-10, convolutional networks would normally be a more appropriate architecture and would likely give better absolute performance. However, the goal here is not to maximize benchmark accuracy, but to compare perturbation-based learning rules under a common model class. This is also consistent with much of the perturbation-learning literature, where fully connected networks are commonly used to study the behaviour of the learning rule itself. The hidden-layer activation function is sigmoid for the two regression tasks and ReLU for the two classification tasks, while all output layers are linear. The specific layer sizes used for each task are shown in Table 3. All layers are implemented as PyTorch `nn.Linear` layers with bias terms. Since no custom initialization is applied, the weights and biases use the default PyTorch initialization for linear layers, which is Kaiming-uniform-style for the weights with fan-in-dependent uniform initialization for the biases.

Table 3: Model architectures used in the empirical evaluation.

Task	Layer sizes	Hidden activation	Output activation	Biases	Initialization
Sinus regression	8-8-8-1	Sigmoid	Linear	Yes	Kaiming-uniform-style weights; fan-in-dependent uniform biases
California housing	8-128-64-1	Sigmoid	Linear	Yes	Kaiming-uniform-style weights; fan-in-dependent uniform biases
MNIST	784-256-128-10	ReLU	Linear	Yes	Kaiming-uniform-style weights; fan-in-dependent uniform biases
CIFAR-10	3072-512-256-10	ReLU	Linear	Yes	Kaiming-uniform-style weights; fan-in-dependent uniform biases

4.4 Algorithms

The perturbation estimators used in the experiments are the same score-function-style estimators derived in Chapters 2 and 3. This section therefore does not rederive the learning rules, but specifies the implementation choices used in the empirical evaluation. These choices are important because they determine how the theoretical estimators are turned into practical mini-batch training algorithms.

All learning methods are trained using mini-batches. The task-specific mini-batch sizes are given in Table 4. For backpropagation, this is the standard mini-batch SGD update: the MSE loss is averaged over the mini-batch, the gradient is computed by backpropagation, and the parameters are updated with SGD. No momentum, Adam, weight decay, or learning-rate schedule is used. Backpropagation is therefore not listed as a separate algorithm below, since it is the standard SGD implementation of the gradient computation described in Chapter 2. It is included only as the reference method against which the perturbation methods are compared.

For the perturbation methods, independent perturbations are sampled for each example in the mini-batch. Thus, each example produces its own noisy forward pass and its own scalar loss. The parameter update is then obtained by averaging the resulting per-example update estimates across the mini-batch. Sampling independent perturbations per example is important because the batch-mean baseline relies on variation between perturbations inside the same mini-batch. If the same perturbation were shared by the whole batch, subtracting the batch mean would remove the useful scalar learning signal.

Let $\mathcal{B} = \{(x_i, y_i)\}_{i=1}^m$ denote a mini-batch of size m , and let ℓ_i be the noisy loss obtained for example i . The scalar learning signal used for each example is

$$r_i = \bar{\ell}_{\mathcal{B}} - \ell_i, \quad \bar{\ell}_{\mathcal{B}} = \frac{1}{m} \sum_{j=1}^m \ell_j.$$

Thus, perturbations that lead to lower-than-average loss receive a positive learning signal, while perturbations that lead to higher-than-average loss are reversed.

In the theoretical derivation, an independent baseline is required for the usual unbiasedness argument. A leave-one-out mini-batch baseline,

$$\bar{\ell}_{-i} = \frac{1}{m-1} \sum_{j \neq i} \ell_j,$$

would satisfy this condition because the baseline for example i would not include the noisy loss generated by the perturbation of that same example. In the implementation, however, the full mini-batch mean $\bar{\ell}_{\mathcal{B}}$ is used instead. This means that the baseline is not strictly independent of the perturbation for example i , so the estimator is not exactly the same unbiased estimator as the leave-one-out version. This choice is made for computational reasons: the full batch mean is obtained directly from the losses already computed in the noisy forward pass, while a leave-one-out baseline would require a separate baseline value for every example in the mini-batch. Since the self-inclusion effect is of order $1/m$, it is expected to be small for the batch sizes used in the experiments, and the practical benefit of avoiding the additional computation is considered to outweigh this small approximation.

Algorithm 1 Weight perturbation (WP)

Require: Training set $\mathcal{D}$, model parameters $\{W_k, b_k\}_{k=1}^L$, epochs E , mini-batch size B , learning rate η , noise scale σ

```
1: for  $e = 1, \dots, E$  do
2:   for mini-batch  $\mathcal{B} \subset \mathcal{D}$  do
3:      $m \leftarrow |\mathcal{B}|$ 
4:     for sample  $i = 1, \dots, m$  do
5:        $a_{0,i} \leftarrow x_i$ 
6:       for layer  $k = 1, \dots, L$  do
7:         Draw  $\xi_{k,i}^W \sim \mathcal{N}(0, I)$ ,  $\xi_{k,i}^b \sim \mathcal{N}(0, I)$ 
8:          $z_{k,i} \leftarrow (W_k + \sigma \xi_{k,i}^W) a_{k-1,i} + b_k + \sigma \xi_{k,i}^b$ 
9:          $a_{k,i} \leftarrow \phi_k(z_{k,i})$ 
10:      end for
11:       $\ell_i \leftarrow \mathcal{L}(a_{L,i}, y_i)$ 
12:    end for
13:     $r_i \leftarrow \frac{1}{m} \sum_{j=1}^m \ell_j - \ell_i$  for all  $i$ 
14:    for layer  $k = 1, \dots, L$  do
15:       $\Delta W_k \leftarrow \frac{1}{m} \sum_{i=1}^m r_i \xi_{k,i}^W / (\sigma + \epsilon_{\text{num}})$ 
16:       $\Delta b_k \leftarrow \frac{1}{m} \sum_{i=1}^m r_i \xi_{k,i}^b / (\sigma + \epsilon_{\text{num}})$ 
17:       $W_k \leftarrow W_k + \eta \Delta W_k$ ,  $b_k \leftarrow b_k + \eta \Delta b_k$ 
18:    end for
19:  end for
20: end for
```

Algorithm 2 Vanilla node perturbation with fixed pre-activation noise

Require: Training set $\mathcal{D}$, model parameters $\{W_k, b_k\}_{k=1}^L$, epochs E , mini-batch size B , learning rate η , noise scale σ

```
1: for  $e = 1, \dots, E$  do
2:   for mini-batch  $\mathcal{B} \subset \mathcal{D}$  do
3:      $m \leftarrow |\mathcal{B}|$ 
4:     for sample  $i = 1, \dots, m$  do
5:        $a_{0,i} \leftarrow x_i$ 
6:       for layer  $k = 1, \dots, L$  do
7:         Draw  $\xi_{k,i} \sim \mathcal{N}(0, I)$ , set  $s_{k,i} \leftarrow \sigma$ 
8:          $z_{k,i} \leftarrow W_k a_{k-1,i} + b_k + s_{k,i} \xi_{k,i}$ 
9:          $a_{k,i} \leftarrow \phi_k(z_{k,i})$ 
10:      end for
11:       $\ell_i \leftarrow \mathcal{L}(a_{L,i}, y_i)$ 
12:    end for
13:     $r_i \leftarrow \frac{1}{m} \sum_{j=1}^m \ell_j - \ell_i$  for all  $i$ 
14:    for layer  $k = 1, \dots, L$  do
15:       $q_{k,i} \leftarrow r_i \xi_{k,i} / (s_{k,i} + \epsilon_{\text{num}})$  for all  $i$ 
16:       $\Delta W_k \leftarrow \frac{1}{m} \sum_{i=1}^m q_{k,i} a_{k-1,i}^\top$ 
17:       $\Delta b_k \leftarrow \frac{1}{m} \sum_{i=1}^m q_{k,i}$ 
18:       $W_k \leftarrow W_k + \eta \Delta W_k$ ,  $b_k \leftarrow b_k + \eta \Delta b_k$ 
19:    end for
20:  end for
21: end for
```

Algorithm 3 Fan-in scaled node perturbation

Require: Training set $\mathcal{D}$, model parameters $\{W_k, b_k\}_{k=1}^L$, layer input dimensions d_{k-1} , epochs E , mini-batch size B , learning rate η , noise scale σ

```
1: for  $e = 1, \dots, E$  do
2:   for mini-batch  $\mathcal{B} \subset \mathcal{D}$  do
3:      $m \leftarrow |\mathcal{B}|$ 
4:     for sample  $i = 1, \dots, m$  do
5:        $a_{0,i} \leftarrow x_i$ 
6:       for layer  $k = 1, \dots, L$  do
7:         Draw  $\xi_{k,i} \sim \mathcal{N}(0, I)$ 
8:          $s_{k,i} \leftarrow \sigma \sqrt{d_{k-1} + 1}$  ▷ includes the bias input
9:          $z_{k,i} \leftarrow W_k a_{k-1,i} + b_k + s_{k,i} \xi_{k,i}$ 
10:         $a_{k,i} \leftarrow \phi_k(z_{k,i})$ 
11:      end for
12:       $\ell_i \leftarrow \mathcal{L}(a_{L,i}, y_i)$ 
13:    end for
14:     $r_i \leftarrow \frac{1}{m} \sum_{j=1}^m \ell_j - \ell_i$  for all  $i$ 
15:    for layer  $k = 1, \dots, L$  do
16:       $q_{k,i} \leftarrow r_i \xi_{k,i} / (s_{k,i} + \epsilon_{\text{num}})$  for all  $i$ 
17:       $\Delta W_k \leftarrow \frac{1}{m} \sum_{i=1}^m q_{k,i} a_{k-1,i}^\top$ 
18:       $\Delta b_k \leftarrow \frac{1}{m} \sum_{i=1}^m q_{k,i}$ 
19:       $W_k \leftarrow W_k + \eta \Delta W_k, \quad b_k \leftarrow b_k + \eta \Delta b_k$ 
20:    end for
21:  end for
22: end for
```

Algorithm 4 Input-scaled node perturbation (IS-NP)

Require: Training set $\mathcal{D}$, model parameters $\{W_k, b_k\}_{k=1}^L$, epochs E , mini-batch size B , learning rate η , noise scale σ

```
1: for  $e = 1, \dots, E$  do
2:   for mini-batch  $\mathcal{B} \subset \mathcal{D}$  do
3:      $m \leftarrow |\mathcal{B}|$ 
4:     for sample  $i = 1, \dots, m$  do
5:        $a_{0,i} \leftarrow x_i$ 
6:       for layer  $k = 1, \dots, L$  do
7:         Draw  $\xi_{k,i} \sim \mathcal{N}(0, I)$ 
8:          $s_{k,i} \leftarrow \sigma \sqrt{1 + \|a_{k-1,i}\|_2^2}$  ▷ WP-induced pre-activation scale
9:          $z_{k,i} \leftarrow W_k a_{k-1,i} + b_k + s_{k,i} \xi_{k,i}$ 
10:         $a_{k,i} \leftarrow \phi_k(z_{k,i})$ 
11:      end for
12:       $\ell_i \leftarrow \mathcal{L}(a_{L,i}, y_i)$ 
13:    end for
14:     $r_i \leftarrow \frac{1}{m} \sum_{j=1}^m \ell_j - \ell_i$  for all  $i$ 
15:    for layer  $k = 1, \dots, L$  do
16:       $q_{k,i} \leftarrow r_i \xi_{k,i} / (s_{k,i} + \epsilon_{\text{num}})$  for all  $i$ 
17:       $\Delta W_k \leftarrow \frac{1}{m} \sum_{i=1}^m q_{k,i} a_{k-1,i}^\top$ 
18:       $\Delta b_k \leftarrow \frac{1}{m} \sum_{i=1}^m q_{k,i}$ 
19:       $W_k \leftarrow W_k + \eta \Delta W_k, \quad b_k \leftarrow b_k + \eta \Delta b_k$ 
20:    end for
21:  end for
22: end for
```

4.5 Evaluation metrics

In supervised machine learning, the central question is how well a trained model performs on unseen data. The most important empirical quantity is therefore the test loss, since this measures whether the learned function generalizes beyond the examples used for optimization. Training loss is also important, because it shows whether the learning rule is able to reduce the objective on the data it is trained on. These two quantities therefore form the main performance measures in the empirical evaluation.

However, training and test loss provide limited diagnostic information. They show how well a method works, but not why it works or fails. A perturbation-based method may perform poorly because its estimated updates are poorly aligned with the true descent direction, or because the estimates fluctuate strongly between perturbation samples. To better understand the learning curves, we therefore also measure directional alignment and estimator variance. These are natural diagnostics for perturbation-based learning, and they are especially relevant in this thesis because they connect directly to the theoretical results in Chapter 3: the expectation-matching result concerns the average update direction, while the variance theorem predicts a difference in estimator variability. The following sections describe the three metric groups used in the experiments, why they matter, and how they are measured in this thesis.

Table 4: Training and estimator-diagnostic settings for each learning problem.

Task	Train batch	Eval batch	Run epochs	Seeds	Diagnostic checkpoints	Gradient estimates per mini-batch
Sinus regression	64	512	500	10	Epochs 1, 120, and 240 of a 240-epoch BP diagnostic run.	240
California housing	256	4096	200	5	Epochs 1, 25, and 50 of a 50-epoch BP diagnostic run.	200
MNIST	128	1024	400	5	Epochs 1, 10, and 20 of a 20-epoch BP diagnostic run.	100
CIFAR-10	128	1024	80	5	Epochs 1, 25, and 50 of a 50-epoch BP diagnostic run.	10

4.5.1 Training and test performance

Training and test performance measure how well the learning rule solves the supervised learning problem. Training loss shows whether the method can reduce the objective on the data used for optimization, while test loss measures whether the learned model generalizes to unseen data. These are the most important metrics in the empirical evaluation, since minimizing the loss and obtaining good test performance is the central goal of training the network.

In all experiments, the loss is the mean squared error (MSE) between the network prediction and the target. For the regression tasks, this is simply the squared difference between the predicted scalar output and the scalar target, averaged over examples. For the classification tasks, the class labels are represented as one-hot vectors, and the MSE is computed between the 10-dimensional network output and the one-hot target vector. Classification performance is also evaluated using accuracy, where the predicted class is taken to be the output component with the largest value. The number of training epochs, evaluation batch sizes, and random seeds used for each task are shown in Table 4. The loss and accuracy curves show the full training trajectory, plotted as averages across seeds as functions of training epoch. To summarize these curves in tables, we report the best value reached during training for each seed. The summary tables then give the mean $\pm$ standard deviation of these best values across seeds; for example, the reported best test loss is the average of the best test loss reached in each individual seed. The convergence epoch is defined as the first epoch at which the test loss has achieved 90% of the total improvement from the initial test loss to the best test loss observed during training.

These metrics are interpreted in three main ways. First, the best training and test values show the final performance reached by each method. Second, the convergence epoch indicates how quickly the method approaches its best observed performance. Third, the shape of the learning curves gives information about stability: smooth curves indicate stable learning dynamics, while large fluctuations or spikes suggest that the updates are noisy or unstable. Together, these quantities show not only how well each method performs, but also how efficiently and reliably it learns.

4.5.2 Directional alignment

All perturbation-based update rules considered in this thesis are intended to estimate a useful gradient direction from stochastic perturbations and a scalar learning signal. In expectation, these estimators point in the same direction as the gradient of the objective they estimate. This makes it natural to ask how well a finite-sample perturbation estimate aligns with the true gradient in practice. Intuitively, if the estimated update direction is well aligned with the true gradient, the learning rule is likely to reduce the loss efficiently. Conversely, if the estimated gradient frequently points in incorrect directions, learning becomes inefficient and may even increase the loss. A natural way to quantify this alignment is through the cosine similarity between the estimated gradient $\hat{g}$ and the true gradient g :

$$\cos(\hat{g}, g) = \frac{\hat{g}^\top g}{\|\hat{g}\| \|g\|}.$$

Here, $\hat{g}$ denotes the gradient estimator produced by the learning rule, while $g = \nabla_\theta L(\theta)$ denotes the true gradient, computed using backpropagation in the experiments. The metric takes values in $[-1, 1]$, where 1 indicates perfect alignment, 0 indicates orthogonality, and negative values indicate that the estimated update direction points against the true descent direction.

The relevance of directional alignment can be seen from a first-order approximation of the loss change. Let $L(\theta)$ be a differentiable loss function and consider an update

$$\theta^+ = \theta + \Delta\theta, \quad \Delta\theta = -\eta\hat{g},$$

where $\eta > 0$ is the learning rate. A first-order Taylor expansion around θ gives

$$L(\theta^+) = L(\theta + \Delta\theta) \approx L(\theta) + \nabla_\theta L(\theta)^\top \Delta\theta.$$

Writing $g = \nabla_\theta L(\theta)$ and substituting $\Delta\theta = -\eta\hat{g}$, we obtain

$$\Delta L = L(\theta^+) - L(\theta) \approx -\eta g^\top \hat{g}.$$

Using

$$g^\top \hat{g} = \|g\| \|\hat{g}\| \cos(\hat{g}, g),$$

this becomes

$$\Delta L \approx -\eta \|g\| \|\hat{g}\| \cos(\hat{g}, g).$$

Thus, for fixed gradient norms, a higher cosine similarity gives a larger first-order decrease in the loss. Positive cosine similarity corresponds to a locally descending update, zero cosine similarity gives no first-order improvement, and negative cosine similarity corresponds to local ascent. This makes cosine similarity a useful diagnostic for the directional quality of perturbation-based gradient estimates, although the actual loss change also depends on update magnitude, estimator variance, and curvature.

Concretely, for each task, a model is first trained with backpropagation and saved at three checkpoints representing early, middle, and later training; the checkpoint epochs are given in Table 4. Backpropagation is used for this diagnostic so that all perturbation rules are compared at the same parameter values. This makes the comparison between estimators fair: differences in cosine similarity then reflect differences in the update estimates themselves, rather than differences in where each method happens to be during training. Using three checkpoints also tests whether estimator quality changes between the beginning, middle, and later stages of training. At each checkpoint

and for each mini-batch in the training set, the reference direction is the exact backpropagation descent direction

$$u_{\text{BP}} = -\nabla_{\theta} L_{\mathcal{B}}(\theta),$$

flattened over all weights and biases. For each perturbation method, the same frozen parameters and the same mini-batch are used to compute K independent gradient estimates. Each estimate is obtained by drawing new random perturbations and applying the corresponding perturbation update rule to the mini-batch, without changing the model parameters. Thus, for method m , this gives K raw update directions

$$\hat{u}_m^{(1)}, \dots, \hat{u}_m^{(K)}.$$

These are then averaged before computing cosine similarity:

$$\bar{u}_m = \frac{1}{K} \sum_{k=1}^K \hat{u}_m^{(k)},$$

Averaging over K independent gradient estimates makes the result less dependent on a single lucky or unlucky perturbation draw, and therefore gives a more reliable estimate of the expected update direction. However, increasing K also increases memory use and computation time. The task-specific values of K , listed as gradient estimates per mini-batch in Table 4, are therefore smaller for the larger image models. The update directions are stored before multiplication by the learning rate, so the cosine comparison measures directional quality independently of the learning-rate choice. The batch-level cosine similarity is then computed as

$$\cos(\bar{u}_m, u_{\text{BP}}) = \frac{\bar{u}_m^{\top} u_{\text{BP}}}{\|\bar{u}_m\| \|u_{\text{BP}}\|}.$$

This value is averaged over all mini-batches in one full pass through the training set and over all random seeds, using 10 seeds for the sinus task and 5 seeds for the other tasks because of their higher computational cost. In the main results, the three checkpoint values are also averaged to give one final cosine value per method and dataset. The summary tables report the mean $\pm$ standard deviation of this aggregate cosine across seeds. The checkpoint-specific cosine values are also saved and plotted separately, but they are moved to the appendix because they do not change the main interpretation and would otherwise add many extra plots to the results chapter.

4.5.3 Estimator variance

Directional alignment measures whether a gradient estimator points in a useful direction, but it does not capture how much the estimate fluctuates between perturbation samples. In perturbation-based learning, the gradient estimate is obtained from random perturbations, so two perturbation samples at the same parameter values can produce different update directions. Even if the estimator is well aligned with the true gradient on average, large fluctuations can make learning inefficient. Variance is also central to the theoretical contribution of the thesis, where IS-NP was shown to have lower or equal coordinate-wise variance than WP under the matched perturbation construction. Estimator variance is therefore a natural diagnostic to measure empirically. Before describing how it is measured, we first show how variance affects the expected change in loss.

The relevance of estimator variance can be seen from a second-order approximation of the loss change. Let $L(\theta)$ be twice differentiable and consider the update

$$\theta^+ = \theta + \Delta\theta, \quad \Delta\theta = -\eta\hat{g},$$

where $\eta > 0$ is the learning rate. A second-order Taylor expansion around θ gives

$$L(\theta^+) \approx L(\theta) + \nabla_{\theta} L(\theta)^{\top} \Delta\theta + \frac{1}{2} \Delta\theta^{\top} H \Delta\theta,$$

where $H = \nabla_{\theta}^2 L(\theta)$ is the Hessian. Writing $g = \nabla_{\theta} L(\theta)$ and substituting $\Delta\theta = -\eta\hat{g}$, we obtain

$$\Delta L = L(\theta^+) - L(\theta) \approx -\eta g^{\top} \hat{g} + \frac{\eta^2}{2} \hat{g}^{\top} H \hat{g}.$$

Taking expectation over the stochastic gradient estimator, and assuming $\mathbb{E}[\hat{g}] = g$, gives

$$\mathbb{E}[\Delta L] \approx -\eta \|g\|^2 + \frac{\eta^2}{2} g^\top H g + \frac{\eta^2}{2} \text{tr}(H\Sigma).$$

The important point is that estimator variance appears directly in the expected loss change through the term $\text{tr}(H\Sigma)$, where H is the Hessian of the loss with respect to the parameters and Σ is the covariance of the gradient estimator. This means that update noise is not merely random fluctuation around an otherwise good direction; it can directly reduce the expected progress made by an update. In particular, variance is most harmful when it lies in directions where the loss has high curvature, because those directions are weighted more strongly by H . Ideally, we would measure the full curvature-weighted quantity $\text{tr}(H\Sigma)$, but estimating the Hessian and the full covariance matrix is not practical for the repeated perturbation experiments used here. We therefore measure estimator variance itself as a tractable and directly relevant diagnostic of update noise.

Estimator variance is naturally defined through the covariance matrix of the vector-valued gradient estimator. For a gradient estimator $\hat{g}$, this covariance matrix is

$$\Sigma = \mathbb{E}[(\hat{g} - \mathbb{E}[\hat{g}])(\hat{g} - \mathbb{E}[\hat{g}])^\top].$$

When the estimator is unbiased for the gradient $g = \nabla_\theta L(\theta)$, this can be written as

$$\Sigma = \mathbb{E}[(\hat{g} - g)(\hat{g} - g)^\top].$$

This is useful here because the exact gradient can be computed using backpropagation. Instead of estimating variance by subtracting the empirical mean of the perturbation samples, we subtract the exact backpropagation direction. However, Σ is a full covariance matrix over all parameters, which is too large to report directly. We therefore reduce it to a scalar by averaging its diagonal entries. Each diagonal entry is the variance of one parameter update component, corresponding to one weight or bias. The resulting quantity,

$$\frac{1}{P} \text{tr}(\Sigma),$$

is the average variance per parameter.

The measurement procedure follows the same frozen-checkpoint structure as the cosine-similarity calculation, using the diagnostic checkpoints and task-specific values of K listed in Table 4. For each checkpoint, mini-batch, and perturbation method, the exact backpropagation descent direction u_{BP} is computed and flattened over all weights and biases. We then draw K independent perturbation-based gradient estimates,

$$\hat{u}_m^{(1)}, \dots, \hat{u}_m^{(K)},$$

using the same frozen parameters and the same mini-batch. The empirical estimate for one method, checkpoint, and mini-batch is then

$$\hat{V}_m = \frac{1}{K} \sum_{k=1}^K \frac{1}{P} \left\| \hat{u}_m^{(k)} - u_{\text{BP}} \right\|_2^2.$$

Here, the factor $1/P$ averages over parameters, while the factor $1/K$ averages over independent gradient estimates.

As for cosine similarity, this variance estimate is averaged over all mini-batches in one full pass through the training set and over random seeds, with the number of seeds given in Table 4. In the main results, the three checkpoint values are also averaged to give one final variance value per method and dataset. The summary tables report the mean $\pm$ standard deviation of this aggregate variance across seeds. Checkpoint-specific variance plots are also generated and included in the appendix, while the main results focus on the aggregate variance plots.

4.5.4 Aggregate Ranking of Perturbation Methods

The empirical evaluation reports several quantities for each method, including training and test loss, training and test score, convergence speed, directional alignment, and estimator variance. Each

metric captures a different aspect of the learning process, but considering all metrics simultaneously can make it difficult to obtain a compact overview of how the perturbation methods compare. To make the results easier to read, an aggregate ranking table is therefore included for each learning problem.

For each metric, the four perturbation-based methods are ranked from 1 to 4, where rank 1 denotes the best value and rank 4 denotes the worst value. Loss, variance, and convergence epoch are ranked in ascending order, while score and cosine similarity are ranked in descending order. The overall ranking is computed as the average rank across the reported metrics. A final aggregate table also reports the average ranks across all four learning problems, giving a compact summary of the observed ordering between the perturbation methods.

This ranking procedure is only intended as a descriptive summary. It weights all metrics equally and does not distinguish between cases where methods are clearly separated and cases where their numerical values are close. These limitations are mitigated by reporting the full metric values, standard deviations across seeds, and learning curves alongside the ranking tables. The ranking tables should therefore not be read as replacing the detailed results, but as a compact guide to the overall pattern of the reported metrics.

4.6 Hyperparameter selection and experimental protocol

Hyperparameter tuning is important because each method must be given a fair chance to learn. If tuning is not done carefully, the results may reflect the chosen hyperparameters more than the learning rule itself. If the learning rate or perturbation scale is too small, learning can become unnecessarily slow or the loss differences can become too weak to give useful updates. If they are too large, the updates can become unstable or no longer reflect a useful local learning signal. Since the learning rate and perturbation scale interact in ways that are not always obvious or trivial, fair tuning requires a systematic search.

For each learning problem, the dataset split, preprocessing, network architecture, mini-batch size, and evaluation procedure are held fixed across learning rules. The tuned hyperparameters differ between methods:

- **Backpropagation:** learning rate η .
- **Weight perturbation:** learning rate η and perturbation scale σ .
- **Vanilla node perturbation:** learning rate η and perturbation scale σ .
- **Fan-in-scaled node perturbation:** learning rate η and perturbation scale σ .
- **Input-scaled node perturbation:** learning rate η and perturbation scale σ .

Each method is tuned separately for each task.

The tuning procedure was carried out in two phases. The first phase focused on selecting reasonable perturbation scales. This was done before tuning the learning rate, because the learning rate and perturbation scale interact in ways that can make training curves difficult to interpret. For example, slow learning could be caused by a learning rate that is too small, a perturbation scale that is too small, a perturbation scale that is too large, or some combination of these. To separate these effects as much as possible, we first evaluated candidate σ values using the frozen-estimator diagnostics from Section 4.5. These diagnostics measure cosine similarity and estimator variance without applying parameter updates, and therefore do not depend on the learning rate. For each task and perturbation method, the search began with a broad set of five candidate σ values. The candidates were evaluated across the three diagnostic checkpoints and averaged. If the results showed a clear trend, for example that larger σ values improved cosine similarity and reduced variance, the range was shifted accordingly. This process was repeated with progressively narrower ranges until three nearby σ values were selected for each method: a central value with the best overall diagnostic

trade-off, and one slightly smaller and one slightly larger value. These three values were then used as the perturbation-scale axis in the training grid search.

The second phase combined the selected perturbation scales with learning-rate tuning. For each perturbation method, we chose three initial learning rates based on preliminary runs and practical experience with the learning algorithms, giving a 3×3 grid over learning rate and perturbation scale. The first goal in this phase was to check whether the learning-rate range was sensible. If the training curves generally improved when the learning rate increased, the range was shifted upward; if larger learning rates caused instability or worse final performance, the range was shifted downward. This stage was evaluated using actual training and test curves, with attention to convergence speed, final loss, and stability. Once the learning-rate range appeared reasonable, the full 3×3 grid was inspected and refined. The grid was narrowed until the results were reasonably consistent and did not show a clear monotonic trend toward one edge of the grid, such as consistently better performance at the highest learning rate or largest perturbation scale. The aim was to choose a final setting from the interior of a stable region, rather than from a boundary where further improvement might simply require extending the search range. Table 5 shows the final local grids and the selected hyperparameter settings used for the thesis results.

Table 5: Local hyperparameter grids used during tuning. For perturbation methods, the grid is the Cartesian product of the listed learning rates and perturbation scales. The selected setting is the final configuration used for the thesis results and is shown as the central value of the displayed local grid.

Task	Method	Learning-rate grid	σ grid	Selected setting
Sinus	BP	{0.075, 0.100, 0.125}	–	$\eta = 0.100$
Sinus	IS NP	{0.125, 0.150, 0.175}	{0.125, 0.150, 0.175}	$\eta = 0.150, \sigma = 0.150$
Sinus	Fan-in NP	{0.100, 0.1125, 0.125}	{0.075, 0.100, 0.125}	$\eta = 0.1125, \sigma = 0.100$
Sinus	Vanilla NP	{0.0750, 0.0825, 0.0900}	{0.250, 0.300, 0.350}	$\eta = 0.0825, \sigma = 0.300$
Sinus	WP	{0.0550, 0.0675, 0.0800}	{0.200, 0.225, 0.250}	$\eta = 0.0675, \sigma = 0.225$
California housing	BP	{0.075, 0.100, 0.150}	–	$\eta = 0.100$
California housing	IS NP	{0.0200, 0.0270, 0.0350}	{0.080, 0.100, 0.120}	$\eta = 0.0270, \sigma = 0.100$
California housing	Fan-in NP	{0.0150, 0.0175, 0.0200}	{0.045, 0.0525, 0.060}	$\eta = 0.0175, \sigma = 0.0525$
California housing	Vanilla NP	{0.0150, 0.0175, 0.0200}	{0.400, 0.450, 0.500}	$\eta = 0.0175, \sigma = 0.450$
California housing	WP	{0.0110, 0.0135, 0.0160}	{0.080, 0.100, 0.120}	$\eta = 0.0135, \sigma = 0.100$
MNIST	BP	{0.400, 0.500, 0.600}	–	$\eta = 0.500$
MNIST	IS NP	{0.0450, 0.0510, 0.0570}	{0.030, 0.0325, 0.035}	$\eta = 0.0510, \sigma = 0.0325$
MNIST	Fan-in NP	{0.0100, 0.0122, 0.0140}	{0.0080, 0.00875, 0.0095}	$\eta = 0.0122, \sigma = 0.00875$
MNIST	Vanilla NP	{0.00450, 0.00563, 0.00675}	{0.110, 0.120, 0.130}	$\eta = 0.00563, \sigma = 0.120$
MNIST	WP	{0.00300, 0.00375, 0.00450}	{0.025, 0.0275, 0.030}	$\eta = 0.00375, \sigma = 0.0275$
CIFAR-10	BP	{0.075, 0.100, 0.125}	–	$\eta = 0.100$
CIFAR-10	IS NP	{0.0100, 0.0120, 0.0140}	{0.016, 0.0175, 0.01875}	$\eta = 0.0120, \sigma = 0.0175$
CIFAR-10	Fan-in NP	{0.00550, 0.00667, 0.00800}	{0.0050, 0.0055, 0.0060}	$\eta = 0.00667, \sigma = 0.0055$
CIFAR-10	Vanilla NP	{0.00320, 0.00400, 0.00480}	{0.165, 0.190, 0.215}	$\eta = 0.00400, \sigma = 0.190$
CIFAR-10	WP	{0.00120, 0.00160, 0.00200}	{0.0110, 0.0125, 0.0140}	$\eta = 0.00160, \sigma = 0.0125$

5 Results

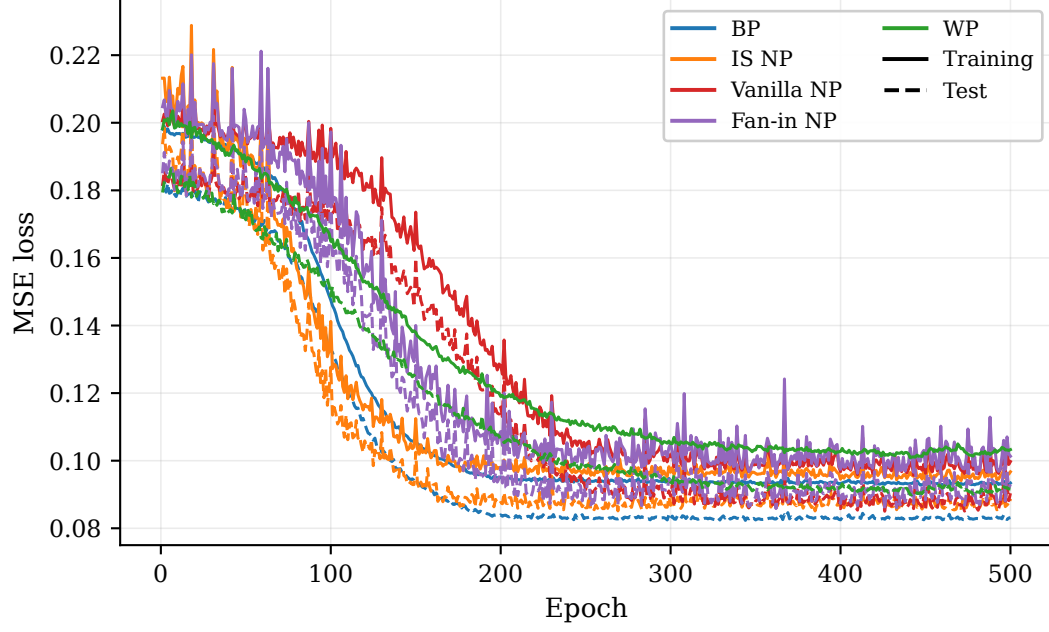
For each learning problem, the main summary table reports the numerical performance and diagnostic metrics. In addition, a ranking table is included for the perturbation methods only. Each perturbation method is assigned a rank from 1 to 4 for each metric, where 1 is best. Lower values are better for loss, convergence epoch, and estimator variance, while higher values are better for score and cosine similarity. The overall rank is the average across the metric-specific ranks, so lower overall scores indicate better performance. After the four task-specific rankings, an aggregate ranking is also reported by averaging the same metric-specific ranks across all four tasks.

5.1 Sinus Regression

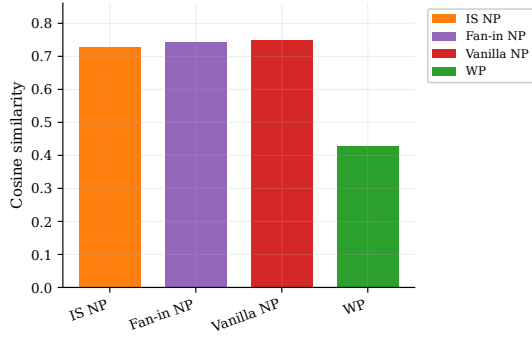
Figure 1 and Table 6 show that all methods were able to learn the sinus regression task well. The node perturbation methods converged to training and test losses close to the backpropagation baseline, with IS-NP and fan-in NP reaching nearly identical loss values. WP ended at a slightly higher loss level, but remained close to the other perturbation methods. IS-NP reached the convergence criterion at approximately the same epoch as backpropagation, while fan-in NP, vanilla NP, and WP converged somewhat later. The standard deviations reported for the best loss and score values are small, indicating little variation across seeds. The fan-in NP curves show somewhat larger fluctuations during training, but without signs of sustained instability. Overall, all perturbation methods worked well as learning algorithms on this task.

The estimator diagnostics show a clearer separation between WP and the node perturbation methods. The three node perturbation methods obtained similar cosine similarity and variance values, with only small differences between them. Vanilla NP had the highest cosine similarity, followed by fan-in NP and IS-NP, while the variance followed the same ordering, meaning that the methods with slightly higher cosine similarity also had slightly higher estimator variance. The differences between the node perturbation methods are therefore small and do not give a clear distinction between them. WP had both lower cosine similarity and higher estimator variance than all node perturbation methods. The reported standard deviations are small for both metrics, indicating that these diagnostic results are consistent across seeds.

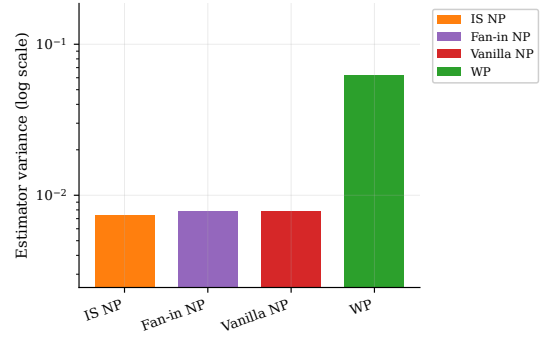
The metric-wise ranking in Table 7 gives the lowest average rank to IS-NP, followed by fan-in NP, vanilla NP, and WP. The clearest distinction is between WP and the node perturbation methods, with WP receiving the lowest rank on every reported metric. Among the three node perturbation methods, the best-ranked method varies across individual metrics, especially for cosine similarity and variance. However, the average ranks separate the methods relatively clearly in the order IS-NP, fan-in NP, and vanilla NP.



(a) Training and test loss.



(b) Cosine similarity.



(c) Estimator variance.

Figure 1: Results for the sinus regression task.

Table 6: Summary metrics for the sinus regression task. Values are mean $\pm$ standard deviation across seeds.

Task	Method	Best train loss	Best test loss	Best train score	Best test score	Score metric	Convergence epoch	Cosine	Variance
Sinus	BP	$0.0925 \pm 3.08\text{e-}4$	$0.0822 \pm 2.24\text{e-}4$	0.529 ± 0.00157	0.536 ± 0.00126	R2	133 ± 27	1.00 ± 0.00	0.00 ± 0.00
Sinus	IS NP	$0.0921 \pm 6.77\text{e-}4$	$0.0825 \pm 8.83\text{e-}4$	0.531 ± 0.00345	0.534 ± 0.00498	R2	120 ± 26	0.728 ± 0.0685	$0.00735 \pm 5.18\text{e-}4$
Sinus	Vanilla NP	$0.0936 \pm 9.59\text{e-}4$	$0.0830 \pm 8.56\text{e-}4$	0.523 ± 0.00489	0.532 ± 0.00483	R2	226 ± 34	0.749 ± 0.0656	$0.00784 \pm 4.38\text{e-}4$
Sinus	Fan-in NP	$0.0931 \pm 5.70\text{e-}4$	$0.0825 \pm 8.34\text{e-}4$	0.526 ± 0.00290	0.534 ± 0.00471	R2	172 ± 29	0.744 ± 0.0634	$0.00787 \pm 4.40\text{e-}4$
Sinus	WP	0.0975 ± 0.00324	0.0865 ± 0.00357	0.504 ± 0.0165	0.512 ± 0.0202	R2	229 ± 58	0.429 ± 0.0913	0.0621 ± 0.00373

Table 7: Metric-wise ranking of perturbation methods for the sinus regression task. Lower ranks are better.

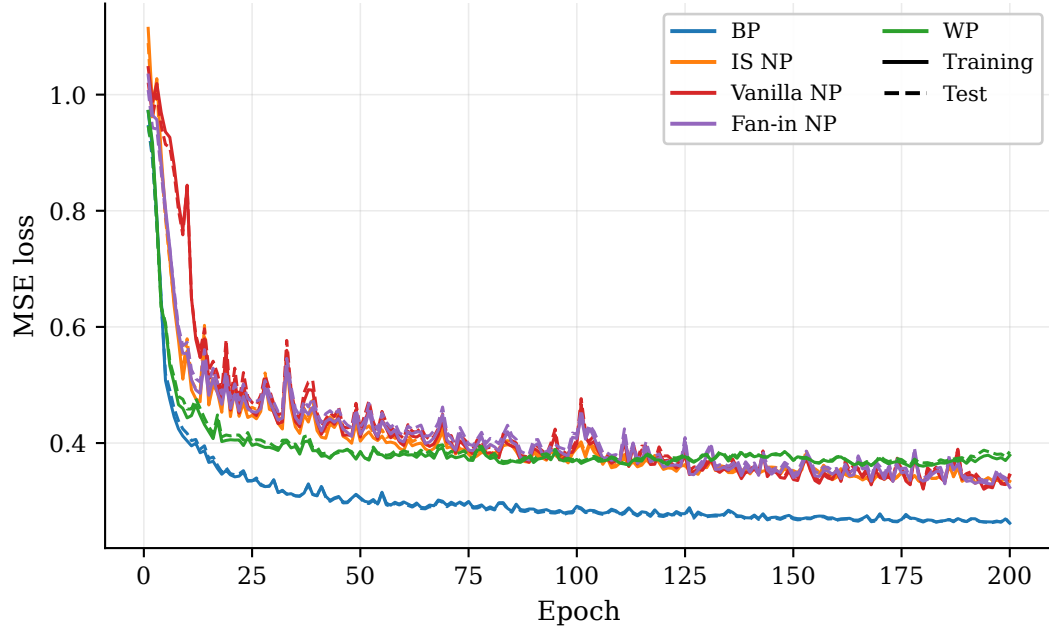
Method	Train loss	Test loss	Train score	Test score	Conv. epoch	Cosine	Variance	Overall
IS NP	1	2	1	2	1	3	1	1.57
Fan-in NP	2	1	2	1	2	2	3	1.86
Vanilla NP	3	3	3	3	3	1	2	2.57
WP	4	4	4	4	4	4	4	4.00

5.2 California Housing Regression

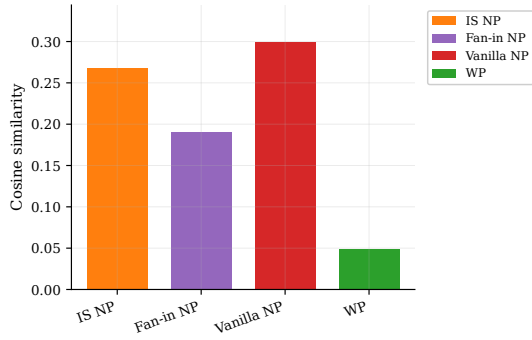
Figure 2 and Table 8 show that all methods were able to learn meaningful predictors on the California housing task, although the perturbation methods converged to higher training and test loss levels than backpropagation. Among the perturbation methods, the three node perturbation methods obtained very similar training and test performance, with no clear separation between them. WP reached the convergence criterion earliest among the perturbation methods, but ended with the highest training and test loss. The standard deviations reported for the best loss and score values are small for BP, IS-NP, vanilla NP, and WP, while fan-in NP shows somewhat larger variation across seeds.

The estimator diagnostics show a clear separation between WP and the node perturbation methods. Vanilla NP obtained the highest cosine similarity, followed closely by IS-NP, while fan-in NP was lower than both. WP had a substantially lower cosine similarity than all node perturbation methods. The variance ranking follows a similar pattern: IS-NP and vanilla NP obtained the lowest estimator variance values, fan-in NP was higher, and WP had the highest variance. The clearest distinction is therefore between WP and the node perturbation methods, but fan-in NP is also separated from IS-NP and vanilla NP on both diagnostic metrics.

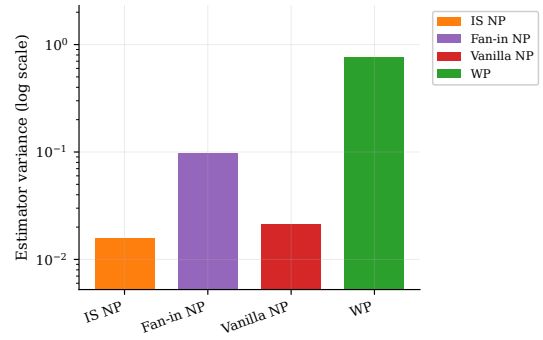
The metric-wise ranking in Table 9 gives the lowest average rank to vanilla NP. Fan-in NP and IS-NP obtain the same average rank, while WP has the highest average rank. The ranking is less uniform than for the sinus task: vanilla NP ranks best on the loss, score, and cosine metrics, IS-NP ranks best on variance, and WP ranks best on convergence epoch. The clearest distinction is again between WP and the node perturbation methods for final performance and estimator diagnostics, while the ordering among the three node perturbation methods depends on the metric considered.



(a) Training and test loss.



(b) Cosine similarity.



(c) Estimator variance.

Figure 2: Results for the California housing regression task.

Table 8: Summary metrics for the California housing regression task. Values are mean $\pm$ standard deviation across seeds.

Task	Method	Best train loss	Best test loss	Best train score	Best test score	Score metric	Convergence epoch	Cosine	Variance
California Housing	BP	0.261 $\pm$ 0.00234	0.261 $\pm$ 0.00253	0.739 $\pm$ 0.00234	0.733 $\pm$ 0.00259	R2	26 $\pm$ 3	1.00 $\pm$ 0.00	0.00 $\pm$ 0.00
California Housing	IS NP	0.324 $\pm$ 0.00471	0.324 $\pm$ 0.00597	0.676 $\pm$ 0.00471	0.668 $\pm$ 0.00613	R2	59 $\pm$ 14	0.268 $\pm$ 0.0346	0.0157 $\pm$ 3.71e-4
California Housing	Vanilla NP	0.312 $\pm$ 0.00495	0.313 $\pm$ 0.00683	0.688 $\pm$ 0.00495	0.678 $\pm$ 0.00701	R2	82 $\pm$ 16	0.299 $\pm$ 0.0214	0.0212 $\pm$ 0.00216
California Housing	Fan-in NP	0.316 $\pm$ 0.0134	0.320 $\pm$ 0.0139	0.684 $\pm$ 0.0134	0.671 $\pm$ 0.0143	R2	78 $\pm$ 17	0.190 $\pm$ 0.0234	0.0969 $\pm$ 0.0150
California Housing	WP	0.340 $\pm$ 0.00967	0.342 $\pm$ 0.00630	0.660 $\pm$ 0.00967	0.649 $\pm$ 0.00647	R2	20 $\pm$ 3	0.0490 $\pm$ 0.00478	0.769 $\pm$ 0.0182

Table 9: Metric-wise ranking of perturbation methods for the California housing regression task. Lower ranks are better.

Method	Train loss	Test loss	Train score	Test score	Conv. epoch	Cosine	Variance	Overall
Vanilla NP	1	1	1	1	4	1	2	1.57
Fan-in NP	2	2	2	2	3	3	3	2.43
IS NP	3	3	3	3	2	2	1	2.43
WP	4	4	4	4	1	4	4	3.57

5.3 MNIST Classification

Figure 3 and Table 10 show that all methods were able to learn meaningful representations for the MNIST classification task, but with clearly separated final performance levels. Backpropagation achieved the highest test accuracy at 97.6

The estimator diagnostics follow the same ordering as the performance metrics. IS-NP obtained both the highest cosine similarity and the lowest estimator variance, followed by fan-in NP, vanilla NP, and WP. WP was separated from the other perturbation methods on both diagnostic metrics, with substantially lower cosine similarity and higher variance. The gap between the node perturbation methods is also visible: fan-in NP is close to IS-NP, while vanilla NP is clearly lower in cosine similarity and higher in variance.

The metric-wise ranking in Table 11 is highly consistent across metrics. IS-NP receives the best average rank and ranks first on every reported metric. Fan-in NP ranks second on all metrics, followed by vanilla NP and WP. The resulting ordering is therefore IS-NP, fan-in NP, vanilla NP, and WP, with only the convergence epoch producing a small deviation between vanilla NP and WP.

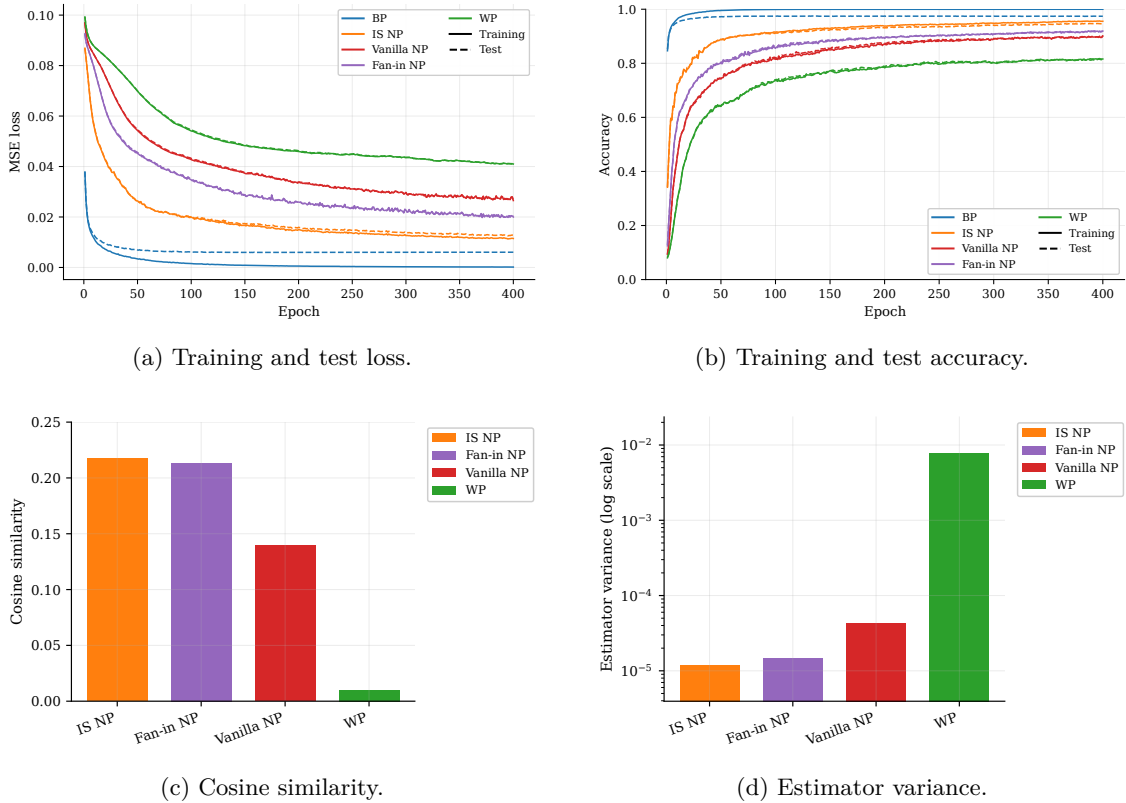


Figure 3: Results for the MNIST classification task.

Table 10: Summary metrics for the MNIST classification task. Values are mean $\pm$ standard deviation across seeds.

Task	Method	Best train loss	Best test loss	Best train score	Best test score	Score metric	Convergence epoch	Cosine	Variance
MNIST	BP	$1.54\text{e-}4 \pm 2.18\text{e-}5$	$0.00587 \pm 1.95\text{e-}4$	$1.00 \pm 1.12\text{e-}4$	$0.976 \pm 6.84\text{e-}4$	Accuracy	22 ± 0	1.00 ± 0.00	0.00 ± 0.00
MNIST	IS NP	$0.0111 \pm 3.32\text{e-}4$	$0.0123 \pm 3.34\text{e-}4$	0.958 ± 0.00151	0.949 ± 0.00263	Accuracy	98 ± 7	0.217 ± 0.0329	$1.18\text{e-}5 \pm 2.14\text{e-}7$
MNIST	Vanilla NP	$0.0259 \pm 6.00\text{e-}4$	$0.0259 \pm 5.61\text{e-}4$	0.905 ± 0.00156	0.904 ± 0.00228	Accuracy	208 ± 15	0.139 ± 0.00865	$4.26\text{e-}5 \pm 3.68\text{e-}7$
MNIST	Fan-in NP	$0.0190 \pm 4.98\text{e-}4$	$0.0192 \pm 4.34\text{e-}4$	0.922 ± 0.00309	0.921 ± 0.00294	Accuracy	173 ± 4	0.213 ± 0.0257	$1.48\text{e-}5 \pm 2.28\text{e-}7$
MNIST	WP	$0.0406 \pm 8.01\text{e-}4$	0.0406 ± 0.00120	0.824 ± 0.00473	0.822 ± 0.00645	Accuracy	193 ± 24	0.00992 ± 0.00127	$0.00787 \pm 9.00\text{e-}5$

Table 11: Metric-wise ranking of perturbation methods for the MNIST classification task. Lower ranks are better.

Method	Train loss	Test loss	Train score	Test score	Conv. epoch	Cosine	Variance	Overall
IS NP	1	1	1	1	1	1	1	1.00
Fan-in NP	2	2	2	2	2	2	2	2.00
Vanilla NP	3	3	3	3	4	3	3	3.14
WP	4	4	4	4	3	4	4	3.86

5.4 CIFAR-10 Classification

Figure 4 and Table 12 show that the gap between backpropagation and the perturbation methods is larger on CIFAR-10 than on the previous tasks. The perturbation methods remain far from the backpropagation baseline and do not learn the task well, although they still achieve accuracies clearly above random guessing. IS-NP obtains the highest test accuracy among the perturbation methods at approximately 36

The estimator diagnostics follow the same ordering as the performance metrics. IS-NP obtains the highest cosine similarity and the lowest estimator variance, followed by fan-in NP, vanilla NP, and WP. WP is again clearly separated from the node perturbation methods, with substantially lower cosine similarity and higher estimator variance. Compared with the regression tasks, the differences between the three node perturbation methods are also more visible on CIFAR-10.

The metric-wise ranking in Table 13 is highly consistent across metrics. IS-NP obtains the best average rank, followed by fan-in NP, vanilla NP, and WP. This ordering appears for all reported performance and estimator-diagnostic metrics. The only exception is convergence epoch, where the ordering is reversed, with WP reaching the convergence criterion earliest and IS-NP latest. Apart from this convergence metric, the ranking gives a clear separation between all four perturbation methods.

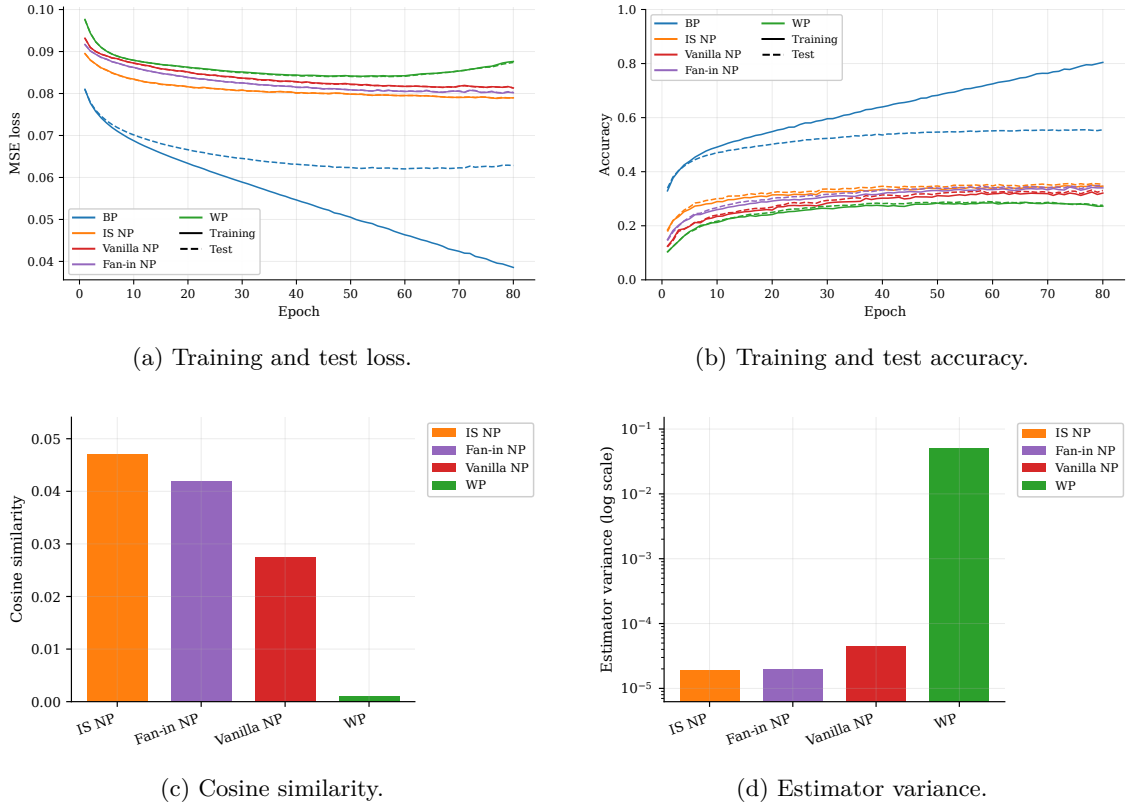


Figure 4: Results for the CIFAR-10 classification task.

Table 12: Summary metrics for the CIFAR-10 classification task. Values are mean $\pm$ standard deviation across seeds.

Task	Method	Best train loss	Best test loss	Best train score	Best test score	Score metric	Convergence epoch	Cosine	Variance
CIFAR-10	BP	$0.0385 \pm 4.14\text{e-}4$	$0.0618 \pm 7.69\text{e-}5$	0.805 ± 0.00513	0.558 ± 0.00303	Accuracy	35 ± 1	1.00 ± 0.00	0.00 ± 0.00
CIFAR-10	IS NP	$0.0788 \pm 1.92\text{e-}4$	$0.0788 \pm 1.28\text{e-}4$	0.356 ± 0.00455	0.362 ± 0.00265	Accuracy	47 ± 3	0.0470 ± 0.00208	$1.87\text{e-}5 \pm 4.35\text{e-}7$
CIFAR-10	Vanilla NP	$0.0812 \pm 1.16\text{e-}4$	$0.0812 \pm 1.47\text{e-}4$	0.332 ± 0.00457	0.337 ± 0.00315	Accuracy	45 ± 2	$0.0275 \pm 7.16\text{e-}4$	$4.44\text{e-}5 \pm 1.00\text{e-}6$
CIFAR-10	Fan-in NP	$0.0798 \pm 2.76\text{e-}5$	$0.0799 \pm 4.49\text{e-}5$	0.350 ± 0.00388	0.354 ± 0.00195	Accuracy	46 ± 3	0.0419 ± 0.00297	$1.97\text{e-}5 \pm 5.35\text{e-}7$
CIFAR-10	WP	$0.0839 \pm 4.94\text{e-}4$	$0.0838 \pm 3.85\text{e-}4$	0.294 ± 0.0109	0.296 ± 0.00637	Accuracy	29 ± 6	$9.92\text{e-}4 \pm 4.38\text{e-}5$	$0.0504 \pm 6.81\text{e-}4$

Table 13: Metric-wise ranking of perturbation methods for the CIFAR-10 classification task. Lower ranks are better.

Method	Train loss	Test loss	Train score	Test score	Conv. epoch	Cosine	Variance	Overall
IS NP	1	1	1	1	4	1	1	1.43
Fan-in NP	2	2	2	2	3	2	2	2.14
Vanilla NP	3	3	3	3	2	3	3	2.86
WP	4	4	4	4	1	4	4	3.57

5.5 Aggregate Ranking of Perturbation Methods

Table 14 summarizes the metric-wise rankings across the four learning problems. IS-NP obtains the lowest average rank on every reported metric and is therefore ranked best overall. It also has the lowest estimator variance on every individual task. Fan-in NP obtains the second-best overall ranking and is clearly ahead of vanilla NP, particularly on the training and test loss metrics. Vanilla NP ranks third overall, while WP obtains the highest average rank on every metric and is clearly the weakest perturbation method in the aggregate ranking.

Table 14: Aggregate metric-wise ranking of perturbation methods across the four learning problems. Values are average ranks across tasks, and lower values are better.

Method	Train loss	Test loss	Train score	Test score	Conv. epoch	Cosine	Variance	Overall
IS NP	1.50	1.75	1.50	1.75	2.00	1.75	1.00	1.61
Fan-in NP	2.00	1.75	2.00	1.75	2.50	2.25	2.50	2.11
Vanilla NP	2.50	2.50	2.50	2.50	3.25	2.00	2.50	2.54
WP	4.00	4.00	4.00	4.00	2.25	4.00	4.00	3.75

5.6 Scaled Input Sensitivity

Scaling the sinus input range from $[-1, 1]$ to $[-5, 5]$ changes the estimator diagnostics differently across methods. IS-NP and WP are only weakly affected by the input scaling, while vanilla NP and fan-in NP degrade substantially. For vanilla NP and fan-in NP, cosine similarity decreases clearly under the scaled input setting, whereas IS-NP remains close to its original level and WP changes comparatively little. The same pattern appears in estimator variance: vanilla NP and fan-in NP show a large increase in variance, while IS-NP and WP remain comparatively stable.

The scaled-input experiment therefore separates the methods more clearly than the original sinus regression task. In the default $[-1, 1]$ setting, all node perturbation methods have relatively high cosine similarity and low estimator variance. Under input scaling, the diagnostics for vanilla NP and fan-in NP worsen, while IS-NP remains the strongest node perturbation method. This result is consistent across both diagnostic metrics shown in Figure 5: the methods that account for the actual effect of input scale, WP and IS-NP, are less sensitive to the change than methods using fixed or layer-width-based node perturbation scales.

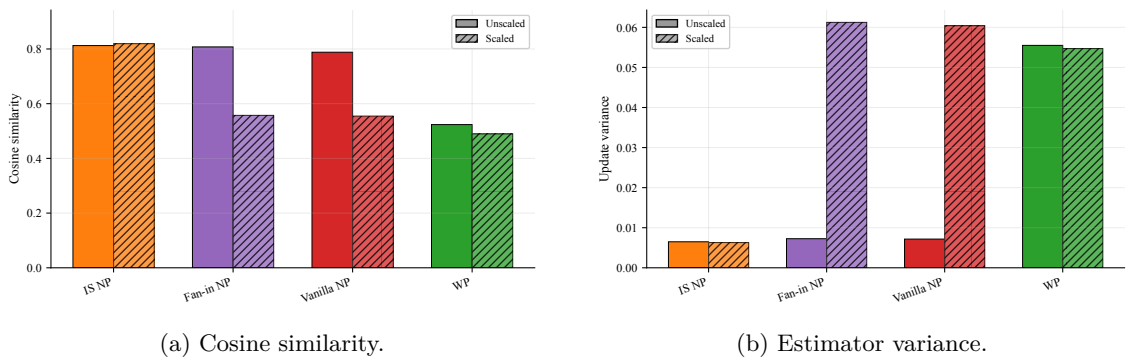


Figure 5: Effect of scaling the sinus input range from $[-1, 1]$ to $[-5, 5]$.

6 Discussion

6.1 Overview of the main findings

The discussion interprets the results in relation to the objectives of the thesis. The first objective, to establish the current state of perturbation-based learning, was addressed in Chapter 2 through the review and comparison of existing perturbation methods. The discussion therefore focuses mainly on the two remaining questions: what the empirical results show about the practical capabilities and limitations of perturbation-based learning, and whether the proposed IS-NP method improves on existing perturbation-based methods.

The results show that perturbation-based methods can learn on all tasks considered, but that their performance relative to backpropagation decreases as task complexity increases. On the simplest regression task, all methods reach similar final performance. On the more difficult tasks, the gap to backpropagation becomes larger, and the differences between perturbation methods become clearer. This supports the view that perturbation methods are viable in simple and moderately difficult settings, but that their noisy gradient estimates become a major limitation as the learning problem becomes more complex.

The results also provide empirical support for the theoretical motivation behind IS-NP. The variance reduction theorem showed that IS-NP is matched to WP in expectation while having no larger coordinate-wise estimator variance. Empirically, IS-NP consistently has much lower variance than WP and generally performs better than the other perturbation-based methods, especially on the classification tasks. The scaled-input experiment further supports the input-scaling argument, as IS-NP is less sensitive than vanilla NP and fan-in NP when activation scales change.

6.2 Practical capability and limits of perturbation-based learning

The empirical results show that perturbation-based learning methods are capable of solving simple and moderately difficult supervised learning problems, but that their performance degrades relative to backpropagation as task complexity increases. On the sinus regression task, all perturbation methods are able to reduce the loss effectively and converge to solutions close to those obtained by backpropagation. This shows that, in sufficiently simple settings, scalar-feedback perturbation methods can provide useful learning signals despite relying on noisy gradient estimates. The California housing task gives a similar overall picture: the perturbation methods are able to learn meaningful predictors on a real tabular dataset, although the differences between methods are less consistent and backpropagation remains the more efficient reference method.

The gap between perturbation-based learning and backpropagation becomes clearer on the image classification tasks. On MNIST, the perturbation methods still learn useful representations, and IS-NP in particular reaches relatively strong performance compared with the other perturbation methods. However, learning is slower than with backpropagation, and the final performance remains worse. This suggests that MNIST is around the level of task complexity where the strongest perturbation method tested here is still capable of producing good empirical results, but where the advantage of exact gradient information already becomes clear.

CIFAR-10 exposes the current limitations of the perturbation methods more strongly. Although the perturbation-based methods are able to learn some structure from the data, their training and test performance remain far below backpropagation under the MLP-based experimental setup used in this thesis. This should not be interpreted as a failure of the experiment. The task progression was deliberately chosen to include cases where the methods begin to break down, because identifying these limits is part of assessing their practical viability. In this setting, CIFAR-10 appears to be beyond what the tested perturbation methods can solve well without additional architectural or optimization improvements.

The estimator diagnostics help explain this trend. As the tasks become more difficult, the perturbation-based estimators generally show weaker directional alignment with the backpropagation gradient

and higher estimator variance. This means that the updates become less reliably directed toward reducing the loss, while also fluctuating more strongly between perturbation samples. Backpropagation avoids this problem by computing the exact gradient directly, which explains why the performance gap grows on harder tasks. Thus, the main empirical limitation of perturbation-based learning is not that it cannot learn at all, but that its noisy gradient estimates become increasingly inefficient as the learning problem becomes more complex.

Overall, the results suggest that perturbation methods are viable for relatively simple tasks and can remain useful on moderately complex problems, but they are not competitive with backpropagation across the full range of tasks considered here. This is consistent with the motivation of the thesis: perturbation methods are attractive because they are simple, local, and biologically plausible, not because they are expected to outperform exact gradient descent in standard artificial neural networks. Their practical value therefore depends on whether their simplicity and biological plausibility can justify the loss in efficiency, and whether theoretically motivated improvements such as IS-NP can extend the range of tasks they are able to solve.

6.3 Empirical support for IS-NP

The empirical results provide strong support for the proposed input-scaled node perturbation method. Across the full set of experiments, IS-NP is generally the best-performing perturbation-based method, particularly on the more complex tasks where differences in estimator quality become most important. This supports the second objective of the thesis, namely to further develop perturbation-based learning through a theoretically motivated modification and assess whether this modification improves practical performance.

The clearest support comes from the comparison with weight perturbation. The theory developed in Chapter 3 showed that IS-NP matches WP in expectation while having no larger coordinate-wise estimator variance. This prediction is strongly reflected in the empirical results. In practice, IS-NP consistently exhibits much lower variance than WP, and this is typically accompanied by better directional alignment with the backpropagation gradient, faster learning, and better final performance. Thus, the theoretical variance reduction is not only a formal result, but also appears to translate into a clear practical advantage.

The comparison with the other node perturbation variants is also encouraging. Overall, the empirical results suggest the ranking

$$\text{IS-NP} > \text{fan-in NP} > \text{vanilla NP} > \text{WP},$$

especially on MNIST and CIFAR-10. On these more demanding tasks, IS-NP converges faster, reaches a better final solution, attains higher cosine similarity, and exhibits lower variance than the other perturbation methods. Fan-in-scaled NP is generally the second-best method, which is consistent with the idea that incorporating some information about input scale is better than using a completely fixed perturbation scale. Vanilla NP is weaker, and WP is generally the weakest method among the perturbation-based baselines.

At the same time, the results also show that these differences are not equally visible on all tasks. On the simpler problems, especially sinus regression and to some extent California housing, the perturbation methods often perform quite similarly in terms of final training and test performance. In such settings, the tasks appear simple enough that all methods are able to find good solutions, even if their estimator quality differs. This likely explains why the ranking is less clear there, and why WP can even appear competitive in some cases, for example in terms of convergence speed on California housing. These exceptions do not contradict the theoretical and empirical story, but rather suggest that when the task is easy enough, differences in estimator quality may matter less for overall optimization performance.

The strongest evidence for the mechanism behind IS-NP comes from the scaled-input experiments. These results show that vanilla NP and fan-in-scaled NP degrade much more severely when the norm of the incoming activations changes substantially, while IS-NP and WP remain comparatively robust. This is exactly what would be expected from the theoretical construction. Vanilla NP

implicitly assumes that a fixed perturbation scale is appropriate across inputs and layers, while fan-in NP adjusts only for layer width. IS-NP instead uses the actual incoming activation norm, and therefore continues to apply perturbations on the correct scale even when those assumptions break down. The empirical results therefore support not only the claim that IS-NP performs better, but also the proposed explanation for why it performs better.

Taken together, the experiments suggest that IS-NP is a meaningful improvement over the most relevant existing perturbation-based baselines considered in this thesis. The advantage is small when the task is very simple, but becomes increasingly clear as the learning problem becomes more demanding or when activation scales vary in ways that challenge the assumptions of simpler NP variants. This does not mean that IS-NP solves the broader limitations of perturbation-based learning, but it does show that the theoretically motivated modification proposed in this thesis pushes this class of methods in a more capable direction.

6.4 Why input scaling matters

The results suggest that input scaling matters because node perturbation is ultimately used to estimate weight-level updates. In vanilla NP, the same pre-activation perturbation scale is used regardless of the magnitude of the incoming activation vector. This is simple, but it ignores how strongly a perturbation at the weight level would affect the corresponding pre-activation. If the incoming activations are small, fixed-scale NP can create a large pre-activation change while assigning only a small weight update, since the update is proportional to the presynaptic activity. If the incoming activations are large, the opposite can happen: the node perturbation may be too small relative to the effect that weight perturbation would naturally induce.

Fan-in-scaled NP partially addresses this problem by scaling the perturbation according to the number of incoming units. This is a reasonable expectation-level correction: under simplifying assumptions, the norm of the incoming activation vector grows with the square root of the layer width. The empirical results also support this interpretation, since fan-in-scaled NP is usually stronger than vanilla NP. However, fan-in scaling still does not observe the actual activation norm. It assumes that layer width is a sufficient proxy for the scale of the incoming vector, which may be inaccurate across samples, layers, datasets, and preprocessing choices.

IS-NP avoids this approximation by scaling the node perturbation using the actual norm of the incoming activations. This makes the perturbation scale sample-dependent and layer-dependent, rather than fixed globally or determined only by layer width. The method therefore preserves the main advantage of node perturbation, namely perturbing a lower-dimensional activity space, while matching the scale of the perturbation more closely to the effect that weight perturbation would have produced at the pre-activation level. This provides a direct explanation for why IS-NP tends to achieve better directional alignment and lower estimator variance than the simpler NP variants.

The scaled-input experiment provides the clearest evidence for this mechanism. When the input range is expanded, the assumptions behind vanilla NP and fan-in-scaled NP become less appropriate, because the activation norms no longer remain on the same scale as before. As expected, these methods show much worse estimator diagnostics and can become unstable. In contrast, WP and IS-NP are much less affected, because both methods naturally account for the scale of the incoming activations: WP through the actual effect of weight noise, and IS-NP through the induced pre-activation noise distribution. This supports the interpretation that IS-NP is not merely better tuned for one particular setting, but is more robust to changes in activation scale.

This robustness is important for the broader motivation of the thesis. In more biologically inspired architectures, it may not be reasonable to assume that all layers, neurons, or input channels operate on carefully normalized scales. A perturbation rule that depends on a fixed global noise scale may therefore be fragile. IS-NP reduces this dependence by adapting the perturbation scale to the local activity entering each neuron. This does not remove the need for hyperparameter tuning, but it makes the perturbation rule less dependent on assumptions about activation magnitudes and layer widths.

6.5 Limitations

The results should be interpreted within the scope of the experimental setup used in this thesis. All experiments use fully connected multilayer perceptrons, even for image classification tasks such as MNIST and CIFAR-10. This choice was made deliberately to keep the comparison focused on the learning rule rather than the architecture. However, it also limits the absolute performance that can be expected on image tasks. In particular, the poor perturbation-based performance on CIFAR-10 should not be interpreted as a general proof that these methods cannot learn complex image tasks, but rather as evidence that they struggle in the MLP setting considered here.

A second limitation is that the empirical comparison depends on the chosen hyperparameter search and training protocol. Perturbation methods are sensitive to both the learning rate and the perturbation scale, and better schedules or optimization tricks could improve performance. For example, reducing the learning rate might prevent instability in some methods during long training runs, while momentum or adaptive normalization could make noisy updates more usable. The hyperparameters used here were chosen to give each method a fair chance under a common experimental framework, but they do not exhaust all possible ways of training these algorithms.

The theoretical result also has a specific scope. The variance reduction theorem compares IS-NP to WP under a matched Gaussian perturbation construction, and shows that IS-NP has no larger coordinate-wise estimator variance while preserving the expected WP update. It does not prove that IS-NP must outperform vanilla NP or fan-in-scaled NP in all settings. The comparison with these methods is motivated by the input-scaling argument and supported empirically in this thesis, but remains dependent on the task, architecture, activation statistics, and hyperparameter choices.

Another limitation is that the experiments focus on standard supervised learning problems. This makes the comparison with backpropagation clear and controlled, but it does not fully capture the settings where perturbation methods may be most biologically relevant. In particular, perturbation-based learning is naturally connected to scalar reward signals, stochastic exploration, and online interaction with an environment. Reinforcement learning tasks, recurrent networks, temporal credit assignment problems, or biologically inspired hardware could therefore reveal different strengths and weaknesses than the feedforward supervised tasks studied here.

Finally, the empirical diagnostics are estimates rather than exact quantities. Cosine similarity and estimator variance are computed from a finite number of perturbation samples at selected stages of training, and therefore provide snapshots of estimator quality rather than a complete description of the learning dynamics. They are useful for explaining the observed trends, especially the difference between WP and IS-NP, but they do not capture every factor that affects optimization. Training performance ultimately depends on the interaction between estimator bias, variance, curvature, learning rate, architecture, and data distribution.

6.6 Future work

A natural direction for future work is to test perturbation-based learning methods on architectures that are better suited to the tasks considered. In this thesis, all methods were evaluated using fully connected multilayer perceptrons in order to isolate the effect of the learning rule. However, convolutional architectures would be more appropriate for image classification tasks such as MNIST and CIFAR-10. Testing IS-NP in CNNs would therefore help determine whether the performance gap to backpropagation is mainly caused by the learning rule itself, or partly by the use of architectures that are poorly matched to the data.

Another important direction is to combine perturbation methods with optimization techniques that reduce the effect of noisy updates. The experiments show that perturbation-based methods can learn, but that their updates become inefficient as task complexity increases. Methods such as momentum, adaptive learning rates, Adam-like normalization, layerwise learning-rate scaling, or variance normalization could make the stochastic estimates more usable in practice. These additions would not change the biological motivation of perturbation learning entirely, but they could substantially improve empirical performance and stability.

Future work should also investigate whether multiple perturbation samples or more structured perturbation schemes can improve estimator quality. The methods studied in this thesis primarily use single-sample perturbation estimates during training, which makes the updates noisy. Averaging several perturbation samples per update would reduce variance, although at increased computational cost. More structured approaches, such as layerwise perturbations, blockwise perturbations, antithetic sampling, or control variates, could provide better trade-offs between estimator quality and computational efficiency.

The theoretical analysis could also be extended beyond the setting considered here. The variance reduction result shows that IS-NP is a Rao–Blackwellized version of WP under a matched Gaussian perturbation construction. However, the comparison between IS-NP and other NP variants remains primarily empirical. A useful direction would therefore be to analyze more precisely when input-scaled node perturbation should outperform fixed-variance or fan-in-scaled NP, and whether similar variance-reduction arguments can be developed for other perturbation distributions, baselines, or network architectures.

Finally, perturbation methods should be tested in settings that are closer to their biological motivation. Supervised feedforward learning provides a controlled benchmark, but perturbation methods are especially natural in settings with scalar reward signals, stochastic exploration, and online interaction. Reinforcement learning, recurrent networks, temporal tasks, neuromorphic systems, or bio-hybrid architectures would therefore be important test cases. These settings may reveal whether the simplicity of perturbation-based learning can become an advantage in architectures where backpropagation is difficult or impossible to apply.

7 Conclusion

Conclusion paragraph 1: Restate motivation and thesis aim.

Conclusion paragraph 2: Objective 1: literature review clarified the perturbation-learning landscape.

Conclusion paragraph 3: Objective 2: theory contribution, IS-NP, expectation matching, lower variance.

Conclusion paragraph 4: Objective 3: empirical results, where methods work/break, IS-NP performance.

Final paragraph: What this means for biologically inspired architectures and future perturbation learning.

Appendix

A Checkpoint-Specific Estimator Diagnostics

This appendix shows the checkpoint-specific cosine similarity and estimator variance plots used to support the aggregate diagnostics in Chapter 4 and Chapter 5. For each learning problem, the diagnostics are shown separately at the three frozen backpropagation checkpoints used in the empirical evaluation.

A.1 Sinus Regression

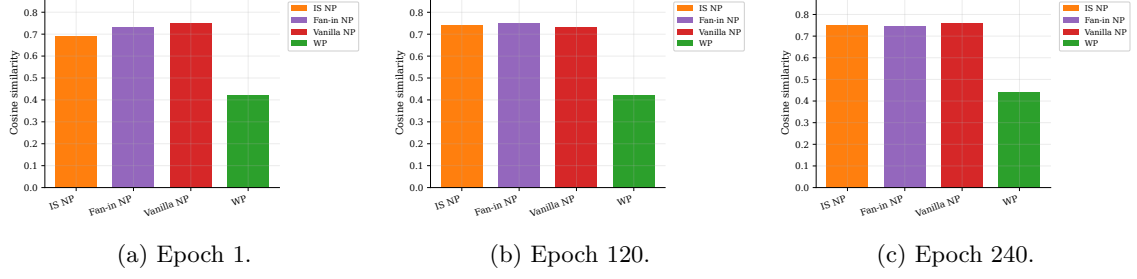


Figure 6: Checkpoint-specific cosine similarity for the sinus regression task.

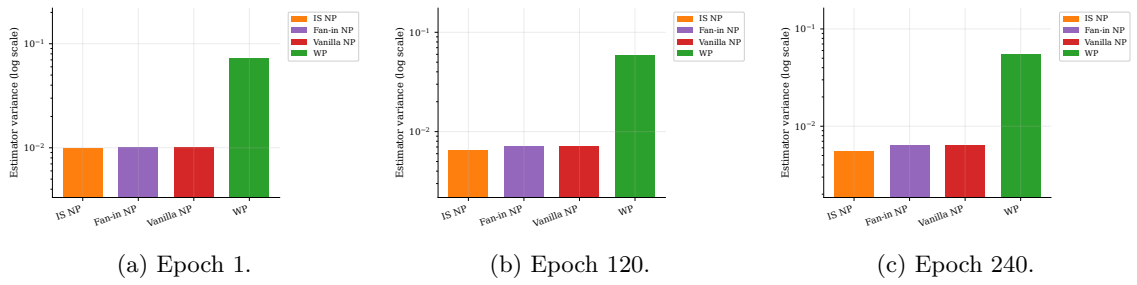


Figure 7: Checkpoint-specific estimator variance for the sinus regression task.

A.2 California Housing Regression

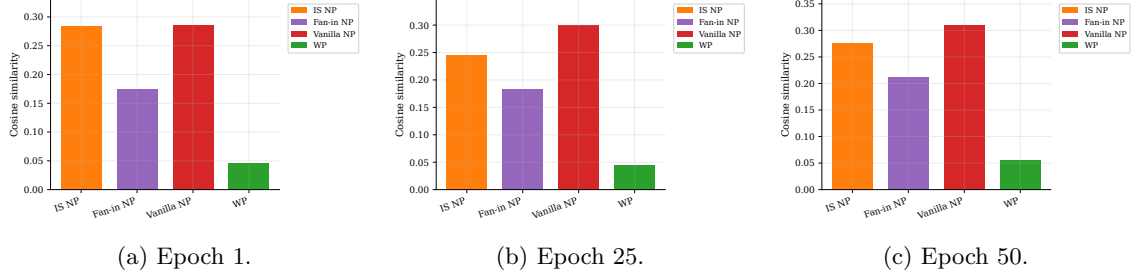


Figure 8: Checkpoint-specific cosine similarity for the California housing regression task.

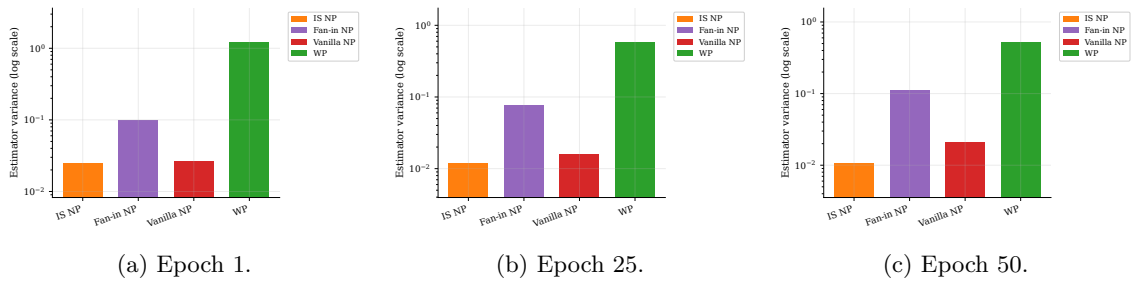


Figure 9: Checkpoint-specific estimator variance for the California housing regression task.

A.3 MNIST Classification

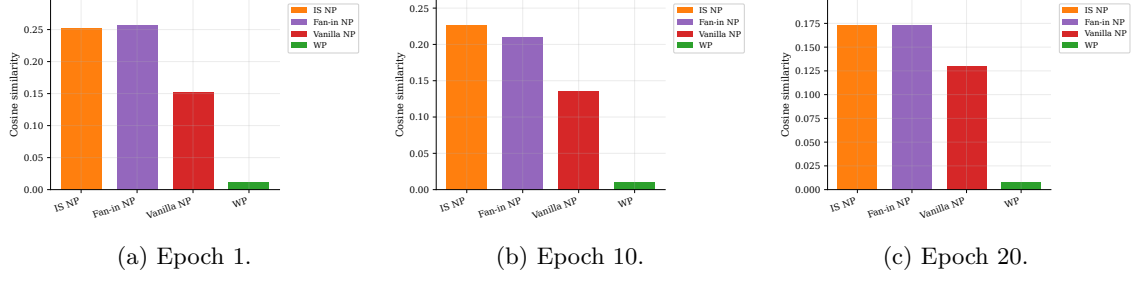


Figure 10: Checkpoint-specific cosine similarity for the MNIST classification task.

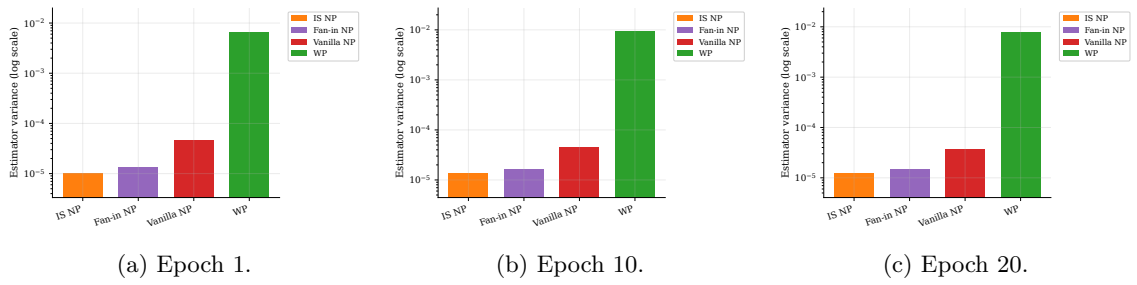


Figure 11: Checkpoint-specific estimator variance for the MNIST classification task.

A.4 CIFAR-10 Classification

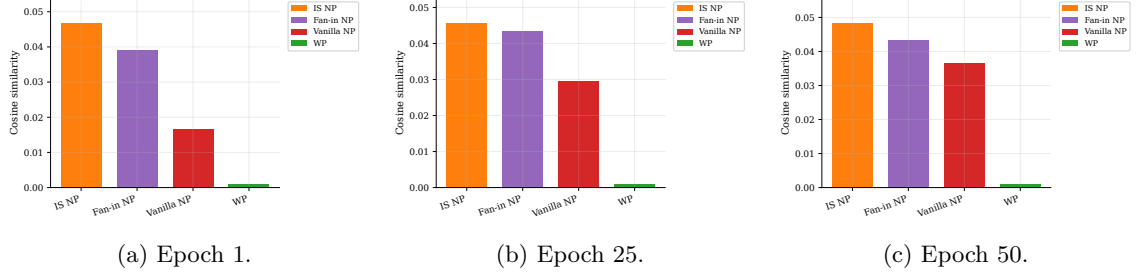


Figure 12: Checkpoint-specific cosine similarity for the CIFAR-10 classification task.

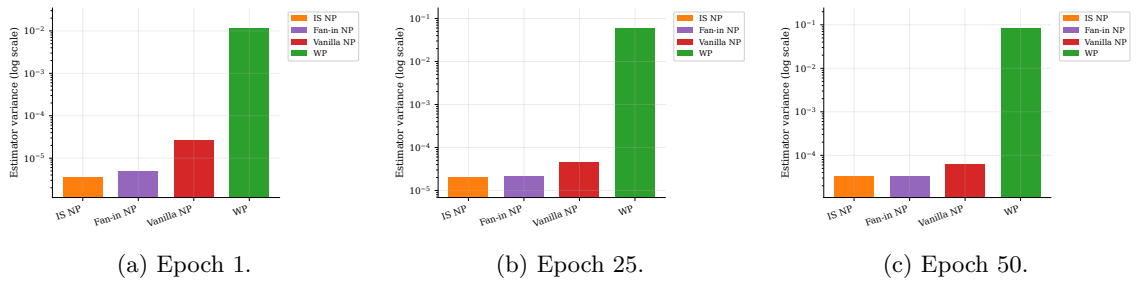


Figure 13: Checkpoint-specific estimator variance for the CIFAR-10 classification task.